

Vysoká škola ekonomická v Praze

Fakulta informatiky a statistiky



EVALUAČNÍ TECHNIKY EFEKTIVNOSTI LLM (LARGE LANGUAGE MODELS)

DIPLOMOVÁ PRÁCE

Studijní program: Data a analytika pro business

Specializace: Praxe v datech a analytice pro business

Autor: Bc. Stanislav Bruch

Vedoucí diplomové práce: Ing. Miroslav Umlauf, MBA

Praha, duben 2025

Poděkování

Na tomto místě bych rád poděkoval svému vedoucímu diplomové práce Ing. Miroslavovi Umlafovi, MBA za jeho cenné rady a připomínky, které mi napomohly k napsání této práce. Zároveň bych chtěl poděkovat Martinu Novákovi a dalším kolegům za podporu.

Abstrakt

Diplomová práce představuje komplexní metodiku pro systematické hodnocení velkých jazykových modelů (LLM) v kontextu firemního nasazení ve společnosti Easy Software. Hlavním cílem práce je vytvoření systematického evaluačního frameworku, který umožní objektivní hodnocení kvality promptů před jejich nasazením do produkčního prostředí softwaru.

Práce propojuje teoretické poznatky o evaluaci LLM s praktickou implementací evaluačního systému založeného na nástroji LangSmith a specializovaných LLM evaluátorech. V rámci práce je navržena a implementována evaluační metodika umožňující systematické porovnávání promptů a modelů na základě definovaných kritérií hodnocení. Zároveň je provedena jejich validace.

Výsledky práce poskytují metodologický základ pro evaluace AI výstupů a modelů v komerční sféře a mohou sloužit jako referenční rámec pro implementaci podobných evaluačních procesů v dalších softwarových organizacích. Práce přispívá k lepšímu porozumění výkonnostním kritériím jazykových modelů a jejich praktické využitelnosti při integraci AI do firemních aplikací.

Klíčová slova

large language models, umělá inteligence, evaluace, prompty, datasety, OpenAI, Llama, LLM judge, LangSmith

Abstract

The thesis presents a comprehensive methodology for the systematic evaluation of large language models (LLMs) in the context of an enterprise deployment at Easy Software. The main goal of the thesis is to develop a systematic evaluation framework that allows for an objective assessment of the quality of the prompts before their deployment in a production software environment.

It combines theoretical knowledge in the field of language model evaluation with the practical implementation of an evaluation system based on the LangSmith tool and specialized LLM evaluators. The thesis proposes and implements an evaluation methodology that enables systematic comparison of prompts and models based on defined evaluation criteria. At the same time, their validation is performed.

The results of the work provide a methodological basis for the evaluations of AI outputs and models in the commercial sphere and can serve as a reference framework for the implementation of similar evaluation processes in other software organizations. The work contributes to a better understanding of the performance criteria of language models and their practical applicability in integrating AI into business applications.

Keywords

large language models, artificial intelligence, evaluation, prompts, datasets, OpenAI, Llama, LLM judge, LangSmith

Obsah

Úvod.....	8
1 Co to jsou Large Language Models	10
1.1 Techniky optimalizace LLM.....	11
1.1.1 Fine-tuning	11
1.1.2 Retrieval-Augmented Generation (RAG).....	13
1.1.3 API model parameter tuning	14
1.1.4 Prompt engineering	14
1.2 Základní techniky promptování	15
2 Evaluační koncepty.....	18
2.1 Základní přístupy k evaluaci LLM.....	18
2.2 Evaluační datasety	18
2.3 Nástroje pro LLM evaluace (LangSmith)	19
2.3.1 Langfuse alternativa	20
2.3.2 Weights & Biases alternativa.....	20
2.3.3 Nástroj pro Easy Software.....	20
2.4 Hodnocení LLM výstupů pomocí evaluátorů	21
2.5 Evaluační kritéria pro LLM na specifických úkolech.....	22
2.5.1 Obecné úkoly (general)	24
2.5.2 Logické uvažování (reasoning).....	24
2.5.3 Programování	25
2.5.4 Matematické úlohy	25
2.5.5 Dlouhý kontext úkolu	25
2.5.6 Multilingvální schopnosti.....	25
3 Charakteristika firmy.....	27
3.1 Popis společnosti	27
3.2 Firemní AI funkcionality.....	28
3.3 Nejčastější způsoby využití AI uživateli	29
3.3.1 Vyhledávání informací.....	29
3.3.2 Překlad textu	29
3.3.3 Oprava gramatiky a pravopisu	29
3.3.4 Odpověď na tiket.....	30
3.3.5 Vytváření úkolů	30
3.3.6 Sumarizace úkolů	30

3.4	Architektura AI aplikace Easy Redmine	30
4	Analýza potřeb firmy pro AI evaluace	33
4.1	Současný stav a výzvy	33
4.2	Firemní požadavky na evaluační proces	33
4.3	Přínosy implementace evaluačního procesu	34
5	Implementace evaluační metodiky do firemního prostředí	37
5.1	Metodologie	37
5.1.1	Výběr evaluačních kritérií LLM judge	38
5.2	Architektura evaluačního systému	38
5.3	Příprava dat a evaluačního datasetu	39
5.4	Implementace evaluátorů pro sumarizační prompt	40
5.4.1	Score Evaluation	40
5.4.2	Evaluátor pro posouzení srozumitelnosti (clarity)	41
5.4.3	Evaluátor pro posouzení věcné správnosti (correctness)	42
5.4.4	Evaluátor pro posouzení stručnosti odpovědi (conciseness)	43
5.4.5	Evaluátor pro posouzení informační retence odpovědi (information retention)	44
5.5	Evaluační prompt pro AI Task Summary	45
5.6	Orchestrace evaluačního procesu	47
5.6.1	Import knihoven	47
5.6.2	Konfigurace a inicializace	48
5.6.3	Spuštění evaluace	48
6	Validace výsledků implementace evaluačního konceptu	51
6.1	Metodika	51
6.2	Výsledky automatických evaluací	52
6.2.1	Srovnání experimentů a modelů	52
6.2.2	Testování konzistence a spolehlivost hodnocení	53
6.3	Validace LLM evaluátorů pomocí lidského hodnocení	54
6.4	Silné a slabé stránky evaluačního konceptu	55
7	Rizika spojená s LLM	57
7.1	Bezpečnost firemních dat	57
7.2	Sleeper agents	57
7.3	Legislativa	59
7.3.1	Legislativní rámec v EU (AI Act)	59
7.3.2	Mezinárodní legislativní rozdíly	60
8	Směry budoucího rozvoje evaluační metodiky a AI řešení	61

8.1 Pokročilé evaluační přístupy pro firemní aplikace.....	61
8.1.1 Dvoustevná evaluace pomocí LLM judge	61
8.2 Agentic RAG.....	62
8.2.1 Návrh architektury Agentic RAG pro firmu	63
8.3 AI agenti.....	63
Závěr	65
Použitá literatura	67
Přílohy	I
Příloha A: Evaluační dataset.....	I
Příloha B: Implementace Score Evaluation: score_evaluation.py	VIII
Příloha C: Implementace evaluátora pro posouzení srozumitelnosti: summarize_clarity_evaluator.py.....	VIII
Příloha D: Implementace evaluátora pro posouzení věcné správnosti: summarize_correctness_evaluator.py	IX
Příloha E: Implementace evaluátora pro posouzení stručnosti odpovědi: summarize_conciseness_evaluator.py.....	X
Příloha F: Implementace evaluátora pro posouzení informační retence odpovědi: summarize_info_retention_evaluator.py	XI
Příloha G: Hlavní skript pro spuštění evaluace: summarization_evaluator.py	XII

Úvod

Velké jazykové modely (LLM) se staly významným technologickým pokrokem v oblasti umělé inteligence a jejich implementace do firemního prostředí nabírá na významu. Od uvedení ChatGPT v roce 2022 došlo k rapidnímu vývoji open-source i proprietárních řešení, která nabízejí různé možnosti využití v komerční sféře. S rostoucím nasazením LLM v produkčním prostředí však vyvstává klíčová otázka zajištění a udržení konzistentní kvality jejich výstupů, zejména při přechodu na novější verze modelů nebo při implementaci nových technik promptování.

Tato diplomová práce vzniká ve spolupráci s firmou Easy Software, která integruje „asistenty“ poháněné umělou inteligencí do svého softwaru. Současnou výzvou je absence systematického přístupu k evaluaci kvality promptů a výkonu různých jazykových modelů. Zatímco existuje řada akademických metrik pro hodnocení LLM, jejich praktická aplikace v produkčním prostředí vyžaduje specifický přístup zohledňující konkrétní potřeby a omezení firemních aplikací.

Hlavním cílem této práce je vytvořit a implementovat proces pro evaluaci kvality promptů, který bude sloužit jako základ pro posouzení kvality AI asistentů. Tento cíl zahrnuje návrh evaluačního frameworku, definici hodnotících kritérií a vytvoření specializovaného datasetu. Součástí je také implementace systému pro automatizované vyhodnocování pomocí LLM.

Dalším cílem práce je vytvořit proces pro systematickou evaluaci LLM modelů, který umožní objektivní porovnání jejich výkonu. Firma se potýká s problémem, kdy má zapotřebí vyhodnotit nově dostupné modely na trhu a evaluovat jejich výkonnost, než je nasadí do produkčního prostředí. Tento proces bude využívat nástroj LangSmith a definovanou sadu standardizovaných promptů pro evaluace.

Metodologicky práce kombinuje analýzu současných evaluačních přístupů s praktickým vývojem a testováním vlastního evaluačního frameworku. Zahrnuje vytvoření specializovaného datasetu bez referenčních výstupů, definici akceptačních kritérií pro LLM judge a implementaci hodnotícího systému v produkčním prostředí.

První kapitola vysvětluje základní koncepty LLM, jejich techniky optimalizace a význam promptů. Kapitola druhá analyzuje současné evaluační metodiky, včetně evaluačních datasetů a nástrojů jako je LangSmith. Třetí kapitola charakterizuje firmu Easy Software a její AI funkcionality v produktu Easy Redmine pro pochopení potřeb pro vyhodnocování. Následující čtvrtá kapitola analyzuje požadavky firmy pro evaluace promptů a LLM modelů. Pátá kapitola detailně popisuje implementaci evaluační metodiky do firemního prostředí, včetně metodiky, s konkrétním zaměřením na sumarizační prompt. Šestá kapitola prezentuje výsledky testování a validace navrženého evaluačního konceptu. Kapitola sedmá odhaluje rizika spojená s využíváním LLM ve firemním prostředí. Závěrečná osmá kapitola nastiňuje možnosti budoucího rozvoje evaluační metodiky a možnosti dalšího rozvoje diplomové práce.

Hlavním přínosem práce je vytvoření prakticky využitelného evaluačního procesu, který pomůže zajistit konzistentní kvalitu AI asistentů při jejich vývoji a nasazení. Výsledky práce poskytnou firmám nástroje a metodiku pro systematické hodnocení jak promptů, tak samotných jazykových modelů, což přispívá k efektivnějšímu vývoji a nasazování AI v softwarových řešeních.

1 Co to jsou Large Language Models

V roce 2013 byl publikován akademický článek na téma „Efficient Estimation of Word Representations in Vector Space“ (Mikolov et al., 2013). Výzkum popisuje techniku NLP (Natural Language Processing), kde je možné převést slova do vektorové podoby. Díky této práci, vědec Tomáš Mikolov položil základy dnešnímu trendu umělé inteligence, jak jej známe dnes. Jedná se o komplexní neuronové sítě trénované na rozsáhlých textových datech, které jsou schopné porozumět a generovat text v daném jazyce. Tyto modely využívají architekturu transformerů, která byla poprvé představena o několik let později v průlomové práci „Attention Is All You Need“ (Vaswani et al., 2017).

Základním principem je předtrénování LLM na velkém množství textových dat, během kterého se model učí porozumět struktuře jazyka, souvislostem a významovým vztahům. Tento proces, známý jako self-supervised learning, umožňuje modelu osvojit si širokou škálu jazykových znalostí bez nutnosti explicitního označování dat. Model se učí předpovídat následující slova v textu na základě předchozího kontextu, což vede k porozumění jazykových vzorů.

Vývoj LLM lze rozdělit do několika etap:

- **První generace modelů GPTs** (Generative Pre-trained Transformer) od OpenAI představené v roce 2018 byly průlomové díky využití architektury transformerů. Tento model ukázal, že trénování na rozsáhlých textových datech následované doladěním na specifických úlohách může vést k výrazně lepším výsledkům než předchozí přístupy. GPT využíval unsupervised předtrénování na velkém korpusu textu a dosáhl pozoruhodných výsledků v řadě úloh zpracování přirozeného jazyka, včetně strojového překladu, odpovídání na otázky a textové klasifikace. Model také demonstroval schopnost řešit úlohy bez specifického tréninku na danou úlohu. Tato první generace GPT-1 měla 117 milionů parametrů a stanovila nový standard pro následující výzkum v oblasti jazykových modelů.
- **Zpřístupnění modelů široké veřejnosti:** Následující modely jako GPT-2 (1,5 miliarda parametrů) a GPT-3 (175 miliard parametrů) přinesly významné zvýšení počtu parametrů a s tím související zlepšení schopností generování textů. Zlomovým momentem bylo uvedení ChatGPT v roce 2022, které ukázalo praktickou použitelnost LLM v každodenních situacích nebo pracovním životě (HF Canonical Model Maintainers, 2022).
- **Rozvoj open-source modelů:** Paralelně s vývojem proprietárních modelů se rozvíjela i open-source komunita. Následovaly projekty jako Falcon či Mistral, které poskytly alternativy k proprietárním modelům a umožnily širší komunitě experimentovat s umělou inteligencí. Také společnost Meta představila sérii open-source modelů zvané Llama, zpřístupnila technologii komunitě i pro komerční využití. Tento trend vedl k vzniku množství specializovaných modelů optimalizovaných pro konkrétní použití nebo jazyky. Nejvíce z nich jsou volně

dostupné na platformě Hugging Face, která slouží jako knihovna pro jednotlivé modely.

Současné LLM se vyznačují několika charakteristikami. První vlastností je škálovatelnost – modely mohou obsahovat miliardy až biliony parametrů, což jim umožňuje zachytit složité jazykové vzory. Obecně platí úměra, že čím více má model parametrů, tím kvalitnější výstupy dokáže generovat. Příčinou je trénování modelu na větším vzorku dat.

Druhou charakteristikou je schopnost few-shot learningu. Jedná se o machine learning přístup, kdy model dokáže adaptovat své chování na základě malého množství příkladů a dat. Tento přístup je užitečný v situacích, kdy je sběr velkých datových sad nákladný nebo časově náročný a urychluje tak vývoj LLM.

Třetí důležitou vlastností je kontextuální porozumění, které umožňuje modelu zpracovávat informace v širším kontextu a generovat kontextuální odpovědi. Tato schopnost je klíčová pro vytváření uživatelsky relevantních výsledků.

1.1 Techniky optimalizace LLM

Pro optimalizaci výstupů velkých jazykových modelů existuje několik hlavních technik, které umožňují přizpůsobit jejich chování specifickým potřebám.

Mezi nejvýznamnější patří:

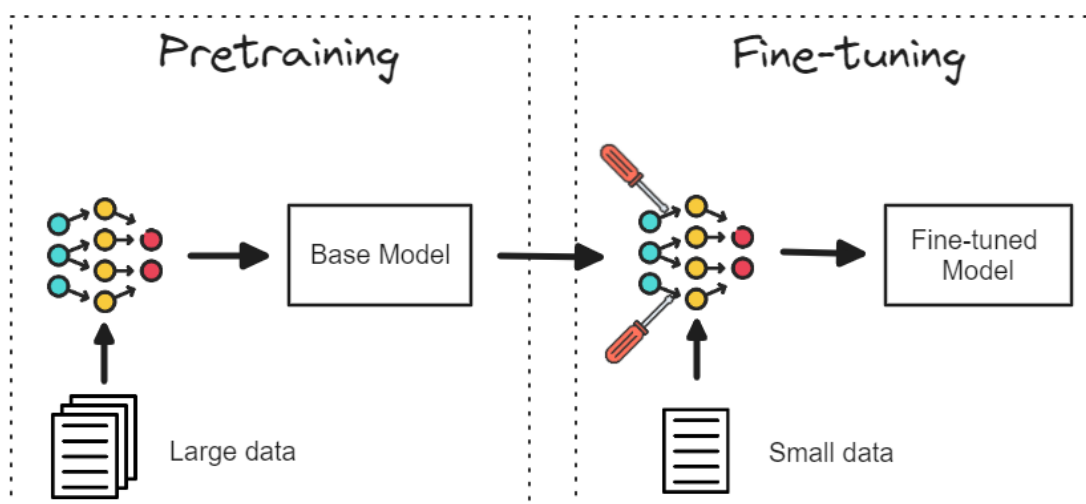
- Fine-tuning,
- Retrieval-Augmented Generation (RAG),
- API model parameter tuning,
- Prompt engineering.

Každá z těchto technik nabízí specifické výhody a je vhodná pro různé případy použití.

1.1.1 Fine-tuning

Fine-tuning představuje proces dodatečného trénování již předtrénovaného modelu na datové sadě pro specifické potřeby. Tato optimalizace umožňuje adaptovat model na konkrétní doménu nebo úlohu při zachování jeho obecných jazykových schopností. Během fine-tuningu se upravují váhy modelu pomocí specializovaného datasetu, což vede k lepším výsledkům v cílové oblasti (Parthasarathy et al., 2024).

Large Language Model



Obr. 1.1 Fine-tuning large language modelu, grafické znázornění (Mahamulkar, 2024)

Například pokud je model dotrénován na právnickou terminologii nebo technickou dokumentaci, dokáže generovat přesnější a relevantnější odpovědi v daném oboru. Toto specializované dotrénování významně redukuje riziko halucinací a faktických nepřesností, protože model získává hlubší porozumění specifickému kontextu a terminologii dané domény.

Proces fine-tuningu však s sebou nese několik výzev. První z nich je výpočetní náročnost na trénování. Dodatečné trénování velkých jazykových modelů vyžaduje specializovaný hardware a významný výpočetní výkon, což může být limitující faktor zejména pro menší organizace nebo projekty s omezeným rozpočtem. Například pro modely o velikosti 7–70 miliard parametrů jsou potřeba desítky GPU a stovky GB RAM, což představuje značné finanční i technické nároky (Parthasarathy et al., 2024).

Další nutností je příprava kvalitních dat. Data by měla být selektivně vybrána, očištěna a strukturována tak, aby reprezentovala požadovanou doménu znalostí a současně obsahovala dostatečnou variabilitu pro efektivní učení modelu. Nekvalitní nebo nevhodně připravená data mohou vést k zhoršení výkonu modelu nebo k nežádoucímu chování. Často označováno jako „halucinace“ modelu.

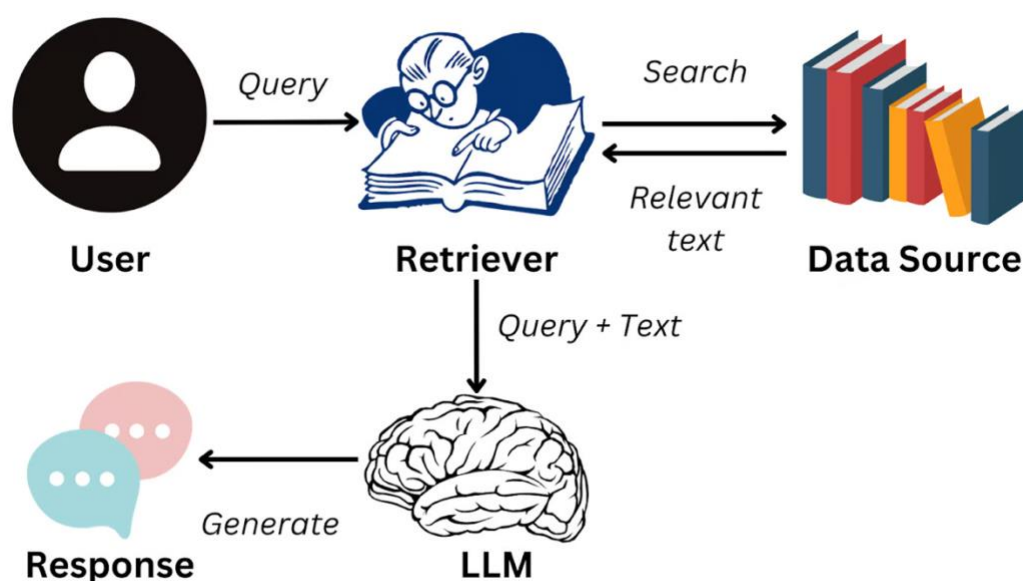
Významným omezením je také skutečnost, že fine-tuning není přenositelný mezi různými modely. To znamená, že i když firma úspěšně aplikuje fine-tuning na jednom modelu, nemůže tyto upravené váhy jednoduše přenést na novější verzi modelu nebo na model od jiného poskytovatele. S každou novou verzí modelu je nutné celý proces fine-tuningu zopakovat, což představuje dodatečné náklady a časovou investici.

V kontextu rychle se vyvíjejícího pole jazykových modelů se techniky fine-tuningu musí neustále adaptovat na nové architektury a verze modelů. Co fungovalo u starší verze modelu nemusí být optimální nebo dokonce možné u novější verze. Tato dynamická povaha vývoje

LLM vyžaduje kontinuální aktualizaci znalostí a postupů v oblasti fine-tuningu. Nicméně jedná se o užitečnou techniku, jak zlepšit výstupy, aniž by bylo zapotřebí natrénovat celý model od základu.

1.1.2 Retrieval-Augmented Generation (RAG)

RAG představuje technologii, která kombinuje LLM s externí vektorovou databází. Jedná se o kontextovou optimalizaci modelu, aniž by došlo k přetrénování modelu. Na rozdíl od fine-tuningu tento přístup nevyžaduje úpravu vah samotného modelu, ale namísto toho rozšiřuje kontext dotazu o relevantní informace získané z dokumentů nebo databází.



Obr. 1.2 Retrieval-Augmented Generation, grafické znázornění (Selvaraj, 2025)

Implementace RAG architektury vyžaduje několik technických komponent. První komponentou je vektorová databáze, která slouží k ukládání dokumentů ve formě vektorových reprezentací (embeddings). Populární volbou jsou databáze jako Chroma nebo Pinecone, které jsou optimalizované pro tento účel. Tyto embeddingy jsou vytvářeny pomocí specializovaných modelů, které převádějí textová data do číselných vektorů zachycujících sémantický význam textu. Dále je potřeba embedding model pro převod textů na vektory (vector embeddings), přičemž často používanými jsou modely od OpenAI.

Proces zpracování dotazu v RAG systému probíhá v několika krocích. Nejprve je uživatelský dotaz převeden na vektorovou reprezentaci stejným způsobem jako uložené dokumenty. Následně systém provede vyhledávání ve vektorové databázi na základě sémantické podobnosti mezi vektorem dotazu a uloženými dokumenty. Nalezené relevantní části dokumentů jsou poté připojeny ke kontextu původního dotazu před jeho odesláním do jazykového modelu.

Nachází uplatnění v různých aplikacích vyžadujících přesné a aktuální informace. V podnikovém prostředí se často využívá pro chatboty, pracující s firemní dokumentací.

Systém umožňuje pracovat s nejnovějšími daty bez nutnosti přetrénování modelu, protože nové informace lze snadno přidat do vektorové databáze.

Nicméně jsou zde i limity, jelikož systém vykazuje vyšší latenci při generování odpovědi kvůli nutnosti vyhledávání v databázi. Také kvalita odpovědi závisí na přesnosti vyhledávacího mechanismu a způsobu, jakým jsou nalezené informace integrovány do promptu. Oproti fine-tuningu lze implementovat pro libovolný model, není třeba investovat značné zdroje do přebudování architektury, naopak nevýhodou je velikost kontextového okna u vstupního promptu (Balaguer et al., 2024).

1.1.3 API model parameter tuning

API model parameter tuning je technika optimalizace výstupů jazykových modelů prostřednictvím nastavení specifických parametrů při volání API od poskytovatele modelu. Jedná se o jednoduchou metodu, jak dodatečně upravit kvalitu výstupu.

Hlavním parametrem je nastavení teploty (temperature), která upravuje kreativitu generovaných odpovědí. Hodnota blízká nule vede k výstupům vhodnějším pro faktické odpovědi, zatímco vyšší číslo umožňuje vyšší míru kreativity.

Společně s teplotou se využívá Top-p sampling pro další kontrolu náhodnosti výstupu přes omezení výběru tokenů. Zatímco teplota ovlivňuje pravděpodobnost všech dostupných tokenů, Top P filtruje možnosti na základě kumulativní pravděpodobnosti. Například Top P s hodnotou 0.8 by znamenalo, že model bude vybírat pouze ze slov, která dohromady tvoří 80% pravděpodobnost, zbylých 20 % nevybere pro generování výstupu (Ravfogel et al., 2023).

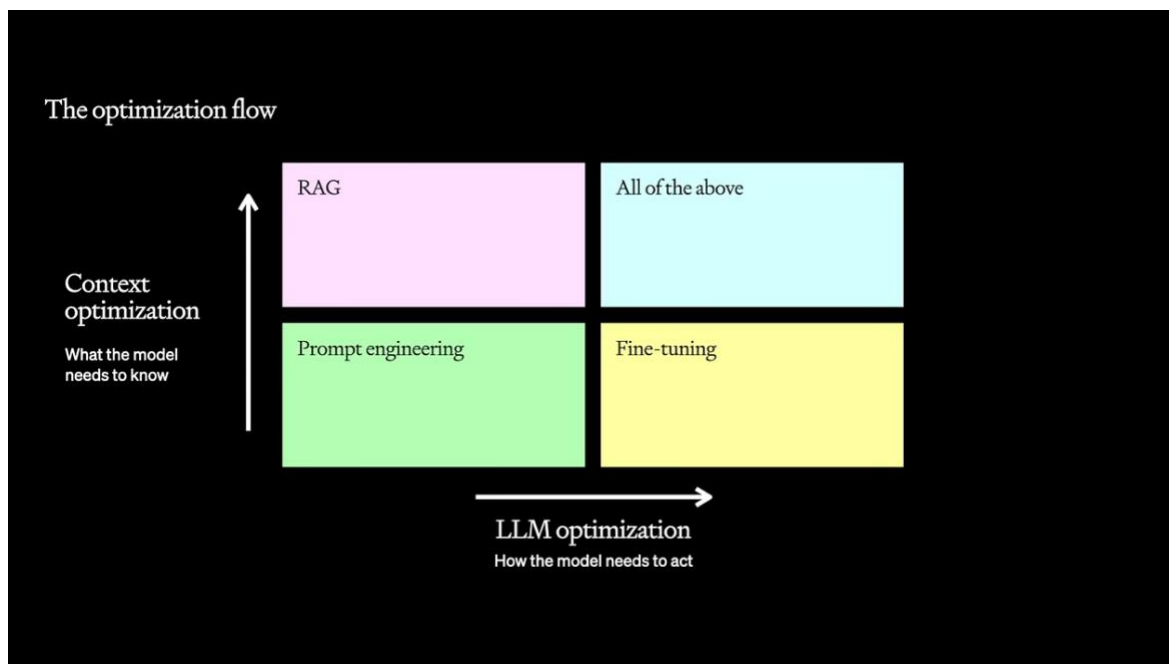
Pro kontrolu délky výstupu slouží parametr Max Length, který určuje strop pro maximální počet generovaných tokenů. Pomáhá tak optimalizovat náklady za každý dotaz. Kvalitu výstupů lze dále ovlivnit pomocí Frequency Penalty a Presence Penalty, které snižují pravděpodobnost opakování slov a podporují rozmanitost jazyka (*OpenAI API Reference*, b.r.).

Hlavní výhodou této techniky je její flexibilita a okamžitá aplikovatelnost pro testování, ačkoliv poskytuje omezenější rozsah optimalizací ve srovnání s komplexnějšími přístupy jako je fine-tuning nebo RAG.

1.1.4 Prompt engineering

Jedná se o nejjednodušší techniku optimalizace vstupních instrukcí pro LLM za pomoci promptu. Zahrnuje úkol, sadu instrukcí, definici role uživatele, které vedou model k požadovanému chování a výstupům. Tato metoda nevyžaduje žádné přetrénování nebo změnu v modelu ani dodatečnou infrastrukturu, což ji činí velmi dostupnou. Efektivní prompt engineering může významně zlepšit kvalitu, konzistenci a relevanci generovaných odpovědí bez technologických bariér. Klíčové je porozumění principům strukturování promptů, včetně použití příkladů (few-shot learning) a explicitních instrukcí. Nicméně pokud uživatel požaduje vyšší kvalitu výstupu, samotná prompt technika nemusí dostačovat

k žádanému výsledku. Nezbyvá než použít další sofistikovanější techniky, jako je zmíněný RAG nebo fine-tuning.



Obr. 1.3 Techniky optimalizace LLM v grafu, osa X vyjadřuje míru optimalizace a osa Y znázorňuje kontextovou optimalizaci (Verma, 2024)

Kombinace těchto metod nabízí komplexní přístup k optimalizaci LLM. Zatímco fine-tuning je vhodný pro přetrénování základního modelu a adaptaci na specifickou doménu, RAG řeší problém s aktuálností a přesností informací díky dynamickému přístupu k externím zdrojům. Prompt engineering poskytuje flexibilní způsob řízení výstupů bez nutnosti technické implementace.

V praxi se často uplatňuje hybridní přístup, který kombinuje výhody jednotlivých metod. Například lze využít fine-tuned model jako základ, který je dále rozšířen o RAG pro přístup k aktuálním datům a celý systém je pak řízen pomocí sofistikovaného prompt engineeringu. Takové řešení může nabídnout optimální rovnováhu mezi přesností, aktuálností a kontrolou nad výstupy. Volba konkrétních metod nebo jejich kombinace závisí na specifických požadavcích projektu a dostupných zdrojích.

1.2 Základní techniky promptování

Kvalitní prompt se typicky skládá z několika částí, které společně vytvářejí kompletní rámec pro interakci s modelem. Základem je popsat nezbytné výchozí informace, co má vykonat za úkol. Dále definice role, která určuje, jakou pozici má model zaujmout – může jít například o experta v dané oblasti, učitele či právníka. Díky definici role se na daný úkol bude dívat správnou optikou. Důležitou součástí jsou také přesné instrukce, které specifikují požadovaný úkol a očekávaný formát výstupu. Může se jednat o požadavky formátování textu, restrikce nebo stylizace textu. Jednotlivé příklady částí promptu:

Role: „Jsi expert na velké jazykové modely.“

Úkol: „Tvým úkolem je vysvětlit, co to jsou LLMs.“

Formátování: „Používej odrážky.“

Restrikce: „Pokud neznáš odpověď, řekni, že neumíš odpovědět na tuto otázku.“

Stylizace: „Vysvětli toto téma jako pětiletému dítěti.“

Případně lze přidat i restrikce, čeho se model nesmí při generování dopustit. Typicky se může jednat o etické nebo nevhodné dotazy. Volitelnou, ale často velmi užitečnou složkou jsou konkrétní příklady, které demonstrují požadovaný formát uživatele nebo styl odpovědi. Poslední komponentou jsou samotná vstupní data (kontext), tedy konkrétní informace, se kterými má model pracovat (Amatriain, 2024).

Pro zvýšení kvality promptů se často využívají pojmenované techniky. Few-shot learning, který začleňuje demonstrační příklady správných odpovědí přímo do promptu. Tento přístup lze ilustrovat na jednoduchém příkladu s hlavními městy:

Q: Jaké je hlavní město Francie?

A: Hlavní město Francie je Paříž.

Model tak získává přímou ukázkou očekávaného formátu odpovědi na otázku. Další technikou je Chain-of-Thought (CoT), která vede model k uvažování krok za krokem, což může pomoci modelu lépe odůvodnit své odpovědi a zvýšit její přesnost (*OpenAI Prompt Engineering*, b.r.). Například při řešení jednoduché matematické úlohy:

„V košíku je 15 jablek. Třetina jablek je červených, zbytek zelených. Kolik je zelených jablek?“

Můžeme modelu v promptu sdělit, aby k postupnému řešení došel pomocí CoT metody. Výstup vypadá následně takto:

„Pojďme postupně vyřešit úlohu. Nejprve víme, že celkem máme 15 jablek. Třetina jablek je červených, tedy 15 děleno 3 je 5 jablek červených. Zbytek jsou zelená jablka, tedy 15 minus 5 se rovná 10 jablek zelených.“

Pro dosažení optimálních výsledků je klíčová specifikace instrukcí. Čím přesnější pokyny model dostane, tím lépe dokáže pochopit požadavky. Pokud se v promptu nenacházejí protichůdné instrukce, opět se navyšuje pravděpodobnost očekávaného výstupu. Důležitá je také přehledná struktura promptu a systematické testování různých variant. Dalšími faktory je volba jazyka odpovědi a styl odpovědi, kdy je třeba přizpůsobit formálnost a odbornost podle kontextu použití.

Samotný prompt engineering však má své přirozené limity. Nemůže například zajistit přístup k aktuálním informacím, což je doména RAG přístupu. Stejně tak nedokáže řešit systematické chyby v trénovacích datech, kde je vhodnější využít fine-tuning. Omezující je

také velikost kontextového okna modelu. Z tohoto důvodu se často přistupuje ke kombinaci různých technik.

Implementace promptu by měla začínat analýzou požadavků od uživatelů, kde definujeme cíl a očekávaný výstup. Následuje fáze návrhu samotného promptu – vytváříme základní strukturu, definujeme roli, instrukce a připravují se demonstrační příklady. Klíčovou součástí procesu je systematické testování různých variant promptu, sběr zpětné vazby a postupné vylepšování na základě získaných výsledků. Celý proces by měl být zakončen důkladnou dokumentací, která zachycuje úspěšné vzory a osvědčené postupy.

2 Evaluační koncepty

S rostoucím nasazením velkých jazykových modelů v produkčním prostředí nastává otázka, jak spolehlivě měřit a zajistit kvalitu jejich výstupů. Zatímco tradiční metriky pro vyhodnocování strojového překladu či sumarizace byly založeny převážně na textové podobnosti s referenčními texty, komplexní povaha moderních LLM vyžaduje sofistikovanější přístupy k evaluaci. Modely dnes řeší široké spektrum úloh od faktického zodpovídání otázek přes logické uvažování až po kreativní psaní, přičemž každá z těchto domén přináší specifické výzvy pro hodnocení kvality. S příchodem instrukcí a kontextového učení je třeba evaluovat nejen správnost, ale i třeba relevanci a užitečnost odpovědi vzhledem k zadanému pokynu. Tato kapitola představuje ucelený pohled na současné metodiky evaluace LLM, jejich principy, silné stránky i omezení.

2.1 Základní přístupy k evaluaci LLM

Evaluace jazykových modelů představuje komplexní problém, který vyžaduje systematický přístup a kombinaci různých metodik. Základním rozdělením evaluačních přístupů je členění na automatické a manuální evaluace. Manuální evaluace jsou procházeny člověkem, u automatických evaluací vstupuje „nezávislé“ LLM, které hodnotí na základě daných kritérií kvalitu odpovědi. Zatímco automatické evaluace nabízejí vyšší rychlost a škálovatelnost, manuální evaluace poskytují hlubší vhled do kvality výstupů, avšak za cenu výrazně vyšší časové a finanční náročnosti.

V oblasti automatických evaluací se historicky využívaly metriky založené na porovnávání krátkých sekvencí slov (tzv. n-gramy). Metriky jako BLEU či ROUGE měří, kolik takových n-gramů se shoduje mezi výstupem modelu a referenčním textem. Tyto metriky však mají svá významná omezení, především v tom, že nedokáží zachytit sémantickou podobnost a kvalitu generovaného textu na vyšší úrovni (Hamdhana et al., 2024). S nástupem velkých jazykových modelů se proto objevily sofistikovanější přístupy využívající právě LLM jako evaluátory, kteří simulují lidské hodnotitele.

2.2 Evaluační datasety

Evaluační datasety slouží primárně pro posouzení výkonu jazykových modelů. Existuje několik typů datasetů, které se používají pro různé účely evaluace a každý má své specifické charakteristiky a použití.

Pro systematickou evaluaci LLM byly vyvinuty společnostmi specializované evaluační benchmark datasety, které se zaměřují na různé aspekty jazykového porozumění a generování. Mezi známé dostupné patří například HumanEval od OpenAI. Tento dataset obsahuje programovací úlohy s testovacími případy, které umožňují automatické vyhodnocení správnosti generovaného kódu (Chen et al., 2021). Další veřejně dostupný

dataset představuje MT-Bench, který obsahuje více úkolové konverzace a umožňuje hodnotit schopnost modelů vést souvislý dialog. Dataset pokrývá různé domény od běžné konverzace přes technické diskuze až po kreativní úlohy. Každá konverzace obsahuje několik výměn, což umožňuje evaluovat i schopnost modelu udržet kontext a konzistenci odpovědí (Bai et al., 2024).

Key-value (KV) datasety jsou nejflexibilnějším typem evaluačních datasetů. Jejich struktura umožňuje ukládat libovolné páry klíčů a hodnot, což je činí vhodnými pro hodnocení řetězců (chains) a agentů, které vyžadují více vstupů nebo vracejí více výstupů. KV datasety jsou zvláště užitečné při hodnocení komplexních systémů. Nevýhodou těchto datasetů je, že evaluace na nich může být složitější, protože je často nutné manuálně specifikovat, jak se mají data připravit pro vyhodnocování.

Syntetické datasety jsou vytvářeny uměle, často pomocí samotných jazykových modelů. Tyto datasety jsou užitečné zejména v případech, kdy nemáme dostatek reálných dat nebo potřebujeme testovat specifické parametry modelu. Při vytváření syntetických datasetů je potřeba zajistit dostatečnou kvalitu dat a relevanci pro danou doménu znalostí. V praxi mohou být cenným doplňkem k reálným datům, zejména v raných fázích vývoje a testování modelů, kdy nejsou dostupné reálná uživatelská data.

2.3 Nástroje pro LLM evaluace (LangSmith)

LangSmith je specializovaná platforma vyvíjená autory frameworku LangChain, která poskytuje komplexní řešení pro sledování a vyhodnocování výkonu LLM aplikací. Její hlavní výhodou je úzká integrace s ekosystémem LangChain, což umožňuje vývojářům jednoduše získat přehled o fungování LLM řetězců. Platforma automaticky zaznamenává všechny vstupy, výstupy a mezikroky zpracování – tzv. traces, které obsahují historii API volání jazykových modelů, včetně metadat (LangSmith, 2023).

Pro evaluace nabízí LangSmith několik přístupů. Populární funkcí je tzv. auto-evaluation, která umožňuje automatické hodnocení generovaných odpovědí buď pomocí jiného LLM modelu v roli „soudce“ (LLM-as-a-judge, více v další podkapitole), nebo prostřednictvím vlastních testovacích funkcí.

K dispozici je také řada předpřipravených evaluátorů a šablon. Vývojáři dále mohou definovat vlastní evaluační datasety obsahující testovací případy a poté sledovat, jak různé verze modelů či promptů na těchto datasetech performují.

LangSmith také plně podporuje sběr zpětné vazby od uživatelů, což umožňuje kombinovat automatické evaluace s uživatelským hodnocením. Tato kombinace je cenná pro komplexní situace, kde některé parametry (například kreativita nebo stylistika) nelze snadno měřit automatizovaně.

Nástroj dále poskytuje metriky týkající se latence a nákladů na volání API, což pomáhá optimalizovat ekonomickou stránku nasazení LLM řešení.

Z hlediska nasazení je LangSmith primárně nabízen jako cloudová SaaS nebo existuje i již nepodporovaná self-hosted varianta. Pro organizace s přísnějšími požadavky na ochranu dat je k dispozici enterprise plán. Jedná se o proprietární řešení, které není dostupné jako open-source software.

2.3.1 Langfuse alternativa

Na trhu existují alternativy k LangSmithu, které nabízejí podobné funkcionality s odlišnými důrazy. Jednou z nich je Langfuse, open-source platforma pro sledování a hodnocení LLM aplikací. Na rozdíl od LangSmithu je Langfuse dostupný pod licencí MIT, což umožňuje jeho snadné self-hosted nasazení pro komerční účely (Langfuse, 2024).

Langfuse nabízí podobné možnosti jako LangSmith – podrobné trasování LLM, vizualizaci procesních toků a různé formy evaluace výstupů. Významnou výhodou je širší kompatibilita, zahrnující přímou integraci s OpenAI API, LangChain, LlamaIndex a dalšími frameworky. Langfuse také poskytuje integrovaný Playground pro experimentování s prompty, což usnadňuje iterativní vývoj. Nevýhodou oproti LangSmithu může být méně propracované vestavěné evaluační metriky a celkově mladší ekosystém.

2.3.2 Weights & Biases alternativa

Další alternativou je Weights & Biases (W&B), zavedená platforma pro sledování ML experimentů, která rozšířila svou funkcionalitu o podporu LLM aplikací (Boorman, 2023). W&B nabízí modul „prompts“ specializovaný na práci s řetězcí LLM volání a prompt engineeringem. Platforma poskytuje podobné funkce s trasováním jako předchozí nástroje, ale vyniká zejména v oblasti vizualizací a týmové spolupráce.

Výhodou W&B je jeho univerzálnost – v jednom prostředí je možné sledovat jak LLM experimenty, tak klasické ML projekty, což je ideální pro týmy pracující napříč různými typy AI. Platforma nabízí flexibilní evaluační framework založený na kombinaci testovacích datových sad a hodnotících funkcí. Bohaté vizualizační možnosti a podpora pro vytváření sdílených reportů usnadňují komunikaci výsledků v týmech. Podobně jako LangSmith je W&B komerční SaaS cloudová služba s možností privátního hostingu, nejedná se ale o open-source řešení, jako je to v případě Langfuse.

2.3.3 Nástroj pro Easy Software

Při volbě nástroje pro evaluaci LLM výstupů hrály v případě Easy Software roli následující faktory. LangSmith byl zvolen především díky jeho bezproblémové integraci se stávajícím kódem postaveným na LangChain, což umožnilo rychlou implementaci evaluačního procesu s minimálními změnami v aplikační logice. Také poskytuje nativně předpřipravené evaluační metriky, které dobře pokrývají požadavky na hodnocení kvality odpovědí.

Alternativní řešení jako Langfuse by bylo vhodné zvážit zejména v případě projektů s přísnými požadavky na open-source technologie nebo potřebou vlastnit a kontrolovat veškerou infrastrukturu. Naopak W&B by byl optimální volbou pro organizace, které již

používají tuto platformu pro jiné ML projekty a chtějí mít všechna data konsolidována v jednom prostředí, což není případ firmy Easy Software.

2.4 Hodnocení LLM výstupů pomocí evaluátorů

Pro hodnocení kvality výstupů jazykových modelů se využívají různí evaluátoři. Existují dva hlavní přístupy k evaluaci – hodnocení člověkem nebo automatizované hodnocení přes další jazykový model, který slouží jako hodnotitel.

Evaluace člověkem (human evaluator) představuje spolehlivý standard pro hodnocení kvality výstupů LLM. Člověk dokáže zachytit nuance v textu, které může automatizovaný hodnotitel přehlédnout. Využívají se dvě techniky k lidskému hodnocení. Přímé hodnocení zahrnuje sběr zpětné vazby od dalších hodnotitelů pomocí dotazníků a hodnotících škál. Druhou technikou je srovnávací hodnocení, kde hodnotitel porovnává dva výstupy od různých modelů přímo mezi sebou.

Například OpenAI využívá tento přístup pro hodnocení dvou výstupů z modelu náhodně na svých uživateli. Následně jsou požádáni, aby vybrali lepší výsledek z dvojice generovaných textů.

Dáváš zpětnou vazbu na novou verzi ChatGPT.
Jaké odpovědi jsou lepší? Načtení odpovědí může nějakou dobu trvat.

Odpověď č. 1

If your `/resources` page works initially but results in a 404 error upon refreshing, this is typically due to how Vercel handles client-side routing in Single Page Applications (SPAs). By default, Vercel serves the `index.html` file for the root path and relies on the client-side router (e.g., React Router) to manage navigation. However, when you refresh a page other than the root, Vercel attempts to retrieve that specific path from the server, leading to a 404 error if the route isn't defined server-side.

To address this issue, you can configure Vercel to serve the `index.html` file for all routes, allowing the client-side router to handle them appropriately. Here's how:

- Create or Update the `vercel.json` File:**
 - In the root directory of your project, create a `vercel.json` file (or update it if it already exists) with the following content:
This configuration ensures that all incoming requests are redirected to `index.html`, enabling the client-side router to manage routing.
- Redeploy Your Application:**
 - After adding or updating the `vercel.json` file, redeploy your application to apply the changes.

By implementing this configuration, Vercel will serve the `index.html` file for all routes, allowing your client-side routing to function correctly even after refreshing a page.

DEV Zdroje

Dávám přednost této odpovědi

Odpověď č. 2

The 404 error on page refresh in your Vercel deployment is typically due to the server not recognizing client-side routes managed by frameworks like React. To address this, ensure that your `vercel.json` file contains the appropriate rewrite rules. The configuration should look like this:

```
json { "rewrites": [ { "source": "/(.*)", "destination": "/" } ] }
```

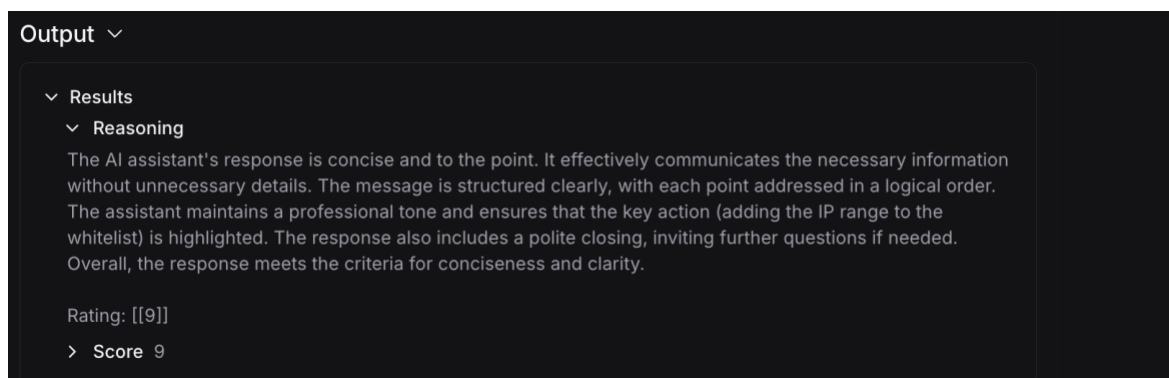
This setup directs all incoming requests to your main `index.html`, allowing your React application to handle routing on the client side. After adding or updating the `vercel.json` file, redeploy your application to apply the changes. This approach has been effective for others facing similar issues. DEV Community

DEV Zdroje

Dávám přednost této odpovědi

Obr. 2.1 Příklad OpenAI AB testingu na uživatelích ve webové aplikaci (ChatGPT, vlastní tvorba)

LLM jako evaluátor (LLM-as-judge) využívá další, nezávislý jazykový model k hodnocení výstupů. Tento přístup může být implementován dvěma způsoby – s referenční odpovědí nebo bez reference v datasetu. Příkladem bez referenčního využití může být LLM evaluátor k detekci nevhodného nebo urážlivého obsahu ve výstupu jiného modelu. Naopak referenční hodnocení porovnává vygenerovaný výstup s předpokládaným příkladem uloženém v datasetu. Zde se například posuzuje, zda má výsledek stejný význam jako referenční výstup. Výhodou využití LLM jako evaluátora je možnost automatizovaného, rychlého a cenově dostupného hodnocení velkého množství příkladů. Nicméně i samotný hodnotící model může mít předsudky. Preference vlastních výstupů, pokud hodnotící model je stejný jako model použitý pro generování výsledků.



Obr. 2.2 Příklad výsledku hodnocení LLM judge evaluátora, obsahuje slovní odůvodnění a skóre kvality (nástroj LangSmith, vlastní tvorba)

Pro komplexní evaluaci LLM výstupů je často nejefektivnější kombinovat oba typy hodnotitelů. Lidské hodnocení může poskytovat hloubkové kvalitativní poznatky, zatímco LLM hodnotitelé umožňují škálovatelné a konzistentní hodnocení napříč velkými datovými sadami. V obou případech je klíčové jasně definovat kritéria hodnocení a zajistit transparentnost a reprodukovatelnost evaluačního procesu.

2.5 Evaluační kritéria pro LLM na specifických úkolech

Evaluace jazykových modelů vyžaduje různé přístupy v závislosti na typu úkolu, který model řeší. Každá oblast má svá specifika, která je třeba zohlednit při výběru vhodných metrik. V této části se zaměříme na šest klíčových oblastí použití LLM a pro každou z nich identifikujeme nejvhodnější evaluační metriky.



Obr. 2.3 Evaluační kritéria pro automatické evaluace ve čtyřech kategoriích (Murugadoss et al., 2024)

Obrázek znázorňuje škálu evaluačních kritérií pro hodnocení LLM modelů ve formě kruhového diagramu, rozděleného do čtyř hlavních dimenzí: integrita (integrity), relevance (relevance), obsah (content) a zapojení (engagement).

Tyto čtyři hlavní dimenze jsou dále členěny na jednotlivá kritéria. Sekce integrita (modrá část) zahrnuje metriky jako logika, robustnost, efektivita, správnost, stručnost, porozumění a schopnost rozumět kontextu.

Sekce relevance (červená část) obsahuje faktickou přesnost, konzistenci, podloženost tvrzení, relevantnost odpovědi a úplnost informací.

Obsah (zelená část) hodnotí komplexnost, plynulost, harmonie, informativnost, hloubku vhledu, kognitivní schopnosti, přirozenost a čitelnost.

Dimenze zapojení (fialová část) posuzuje schopnost modelu překvapit, zaujmout, složitost a empatii.

Celek znázorňuje strukturovaný pohled na mnohorozměrnou povahu evaluace LLM, kde jsou jednotlivé aspekty vzájemně propojeny a společně tvoří komplexní hodnotící rámec. Každá z těchto oblastí vyžaduje specifický přístup k evaluaci, přičemž je často nutné kombinovat několik metrik pro získání komplexního obrazu o schopnostech modelu. V praxi je důležité vybrat metriky, které nejlépe odpovídají konkrétním potřebám.

2.5.1 Obecné úkoly (general)

Pro hodnocení obecné schopnosti modelů generovat kvalitní odpovědi na běžné dotazy jsou klíčové tyto metriky.

Faktická přesnost hodnotí, zda model poskytuje fakticky správné informace a nevymýšlí si nepravdivá tvrzení. Tato metrika je zvláště důležitá pro generování odpovědí na znalostní otázky, kde „halucinace“ mohou výrazně snížit použitelnost modelu.

Relevance měří, zda odpověď modelu přímo adresuje položený dotaz. Irelevantní nebo jen částečně relevantní odpovědi naznačují, že model nedokáže správně interpretovat zadání nebo udržet kontext.

Kompletnost hodnotí, zda model pokryl všechny aspekty dotazu a poskytl dostatečně vyčerpávající odpověď. Příliš stručné nebo neúplné odpovědi mohou uživatele odradit, i když jsou fakticky správné.

Koherence zkoumá, zda je odpověď vnitřně konzistentní a logicky strukturovaná. Nekoherentní odpovědi, byť obsahově správné, snižují důvěryhodnost a použitelnost modelu.

2.5.2 Logické uvažování (reasoning)

Pro úkoly vyžadující logické uvažování a řešení problémů jsou vhodné následující metriky.

Logická robustnost měří schopnost modelu udržet logickou konzistenci v průběhu řešení problému.

Logická správnost hodnotí, zda model dospěl ke správnému závěru na základě dostupných premis a zda jsou jednotlivé kroky uvažování správné.

Logická efektivita zkoumá, zda model dokáže problém vyřešit optimálním způsobem bez zbytečných kroků nebo odbočení.

Chain-of-Thought (v překladu řetězec myšlenek) hodnotí schopnost modelu explicitně artikulovat jednotlivé kroky uvažování, což je klíčové pro složitější problémy vyžadující strukturované myšlení.

2.5.3 Programování

Pro analýzu schopnosti modelů generovat a analyzovat kód jsou důležité tyto metriky:

Funkčnost kódu nám říká, zda vygenerovaný kód funguje podle očekávání a splňuje požadavky zadání. Toto lze testovat automaticky pomocí předem připravených testovacích případů.

Efektivita kódu hodnotí časovou a prostorovou složitost vygenerovaného řešení. Efektivní kód by měl minimalizovat spotřebu výpočetních zdrojů.

Čitelnost kódu zkoumá, zda je kód dobře strukturovaný, komentovaný a zda dodržuje konvence a osvědčené postupy pro daný programovací jazyk.

Bezpečnost kódu je důležitá pro vyhodnocení, zda vygenerovaný kód neobsahuje potenciální bezpečnostní zranitelnosti nebo neošetřené chybové stavy.

2.5.4 Matematické úlohy

Schopnost řešení matematických úloh je jeden z hodnoticích předpokladů pro hodnocení výkonnosti modelu.

Přesnost výsledku hodnotí, zda model dospěl ke správnému numerickému výsledku nebo matematickému závěru.

Metodologická správnost analyzuje, zda model použil správné matematické metody a postupy k řešení problému.

Schopnost formalizace hodnotí, jak dobře model dokáže převést slovní úlohu do matematického zápisu, což je často důležitým krokem v řešení komplexních problémů.

2.5.5 Dlouhý kontext úkolu

Pro práci s rozsáhlými dokumenty a dlouhým kontextem jsou klíčové tyto metriky.

Schopnost vyhledávání informací hodnotí, jak dobře model dokáže identifikovat a extrahovat relevantní informace z rozsáhlého kontextu.

Relevance zkoumá, zda model dokáže udržet fokus na zadaný úkol i při zpracování velkého množství potenciálně rozptylujících informací.

Přesnost měří schopnost modelu kondenzovat rozsáhlé informace do stručných a přesných shrnutí bez ztráty klíčových detailů.

2.5.6 Multilingvální schopnosti

Modely často pracují s různými jazyky, jak při vstupu, tak i při generování odpovědi. Následující metriky jsou vhodné pro posouzení kvality výsledku.

Jazyková přesnost pro gramatickou a stylistickou kvalitu výstupů v různých jazycích, včetně správného použití idiomů a kulturně specifických výrazů.

Zachování významů sleduje, jak dobře model zachovává původní význam a nuance při převodu obsahu mezi různými jazyky.

Kulturní citlivost hodnotí, zda model zohledňuje kulturní kontext a specifika při generování odpovědí v různých jazycích.

3 Charakteristika firmy

Tato kapitola se zaměřuje na popis současného stavu implementace AI funkcionalit a jejich využití v produktivním prostředí. Detailní pochopení těchto produktů je důležité pro následný návrh evaluační metodiky, která musí zohledňovat specifické požadavky jednotlivých případů použití.

3.1 Popis společnosti

Společnost Easy Software je středně velká technologická společnost působící na českém trhu od roku 2006. Firma se specializuje na vývoj a implementaci komplexních podnikových informačních systémů se zaměřením na projektové řízení práce. V roce 2024 společnost rozšířila své produktové portfolio o řešení využívající umělou inteligenci, především v oblasti zpracování úkolů, projektů a tiketů zákaznické podpory. S více než 90 zaměstnanci poskytuje Easy Software své služby klientům od malých a středních podniků až po velké korporace, především v sektorech výroby, IT služeb a automotive.

Významným milníkem v historii společnosti bylo její zapojení do open-source komunity kolem softwaru Redmine. Od roku 2008 se aktivně podílela na vývoji, přispívala vlastními pluginy do hlavní větve projektu a vytvořila řadu rozšíření. Tyto zkušenosti z open-source prostředí významně ovlivnily firemní kulturu a přístup k vývoji software, který staví na principech transparentnosti a sdílení znalostí.

Společnost je známá především díky poskytování serverových služeb, které umožňují klientům plnou kontrolu nad jejich daty a systémy při zachování vysoké úrovně bezpečnosti a spolehlivosti. Také je první firmou na světě, jež nabídla klientům on-premises AI řešení v oblasti projektového řízení.

Hlavním produktem společnosti je Easy Redmine, platforma pro řízení projektů a práce, která integruje několik oblastí do jednoho řešení – hybridní projektové řízení, IT service management a správu zdrojového kódu. Systém podporuje obě metodiky, ať už se jedná o Waterfall nebo agilní řízení projektů, včetně plánování zdrojů a dalších funkcí. Kromě on-premises nasazení softwaru je produkt dostupný i v cloudovém řešení (SaaS) nebo lze nasadit na privátní cloud. Díky otevřenému API a integraci s Make.com lze propojit s tisíci dalšími aplikacemi.

Produkty v oblasti AI jsou rozděleny do tří hlavních kategorií. Personal AI, Project AI a Service AI. Toto rozdělení reflektuje různé kategorie využití v závislosti na roli uživatele a typu požadovaných funkcí. Personal AI slouží pro každodenní práci s úkoly, kde patří funkce typu úprava textu, vyhledávání znalostí v databázi či sumarizace úkolů. Dále Project AI je pomocník pro projektové řízení, ať už se jedná o plánování projektů nebo automatizace projektových statusů pro projektové manažery. Poslední kategorií je Service AI, slouží zejména pro zákaznickou podporu pro rychlejší a přesnější odpovědi klientům.

Celý systém je navržen tak, aby poskytoval konzistentní odpovědi napříč všemi produkty, přičemž využívá jednotnou znalostní bázi organizace nebo různé datové kolekce, přizpůsobené na daný produkt a příklad užití.

3.2 Firemní AI funkcionality

- **Knowledge Assistant:** Představuje základní AI funkci v softwaru Easy Redmine, slouží totiž jako „mozek firemních znalostí“. Systém podporuje kombinaci různých datových zdrojů, například PDF dokumenty, Markdown soubory, webové stránky, znalostní báze a data z úkolů. Po kliknutí na ikonku uživatel zadá otázku a poskytne rychlou odpověď na dotaz uživatele. Výhodou je kontextově relevantní odpověď, podpořená odkazy, odkud asistent čerpal. AI také pomáhá s orientací v samotném produktu, což usnadňuje onboarding nových uživatelů.
- **Write Assistant:** Pomáhá vylepšovat a upravovat text v úkolech a projektech. Na výběr je několik možností využití asistenta. Překlad textu usnadňuje komunikaci mezi uživateli. Funkce vylepšení textu zajišťuje gramatiku a profesionalitu psané komunikace. Asistent je dostupný ve všech textových polích systému, včetně komentářů a aktivuje se automaticky při psaní nebo označení textu.
- **HelpDesk Assistant:** V kontextu zákaznické podpory automatizuje proces odpovídání na tikety. Asistent nejprve analyzuje příchozí dotaz klienta z dat tiketu. Následně na základě historických dat z vyřešených dotazů a znalostní báze generuje návrh odpovědi pro klienta. Funkce tak významně zrychluje reakční dobu podpory a zajišťuje konzistenci odpovědí.
- **AI Project Planner:** Nabízí pro uživatele pokročilou funkcionality, který pomáhá plánovat a vytvářet projekty. Na základě nahraného PDF dokumentu (například smlouva nebo kontrakt) asistent identifikuje akční úkoly v textu a vygeneruje projektový plán. Výstupem je komplexní struktura úkolů ve WBS (Work Breakdown Structure) a Ganttově diagramu, včetně souhrnných informací na projektovém dashboardu.
- **AI Task Summary:** Poskytuje rychlý přehled o dění v úkolech bez potřeby procházet veškerou historii komentářů. Uživatelé si mohou vybrat mezi dvěma formáty výstupu – stručným textem nebo odrážkovým seznamem. Tato funkcionality významně šetří čas při orientaci v úkolech a umožňuje rychlé pochopení aktuálního stavu.

Všechny tyto funkcionality sdílejí společnou znalostní bázi organizace, kterou pohání Wiki technologie, jako je to v případě Wikipedie. Easy Redmine software umožňuje správu uživatelských práv k jednotlivým kolekcím, což zajišťuje bezpečnost a relevanci poskytovaných informací pro různé skupiny zaměstnanců.

Zároveň jsou v produktu dostupné šablony promptů, které mají v sobě definované specifické instrukce, restrikce a roli pro jazykový model podle zvolené AI funkcionality. Tyto šablony lze přizpůsobovat a vytvářet nové podle potřeb uživatele, což zajišťuje flexibilitu a možnost optimalizace pro různé scénáře použití. Zároveň ale vytváří potřebu vyhodnocovat kvalitu promptu.

3.3 Nejčastější způsoby využití AI uživateli

V posledních letech se umělá inteligence stále více integruje do softwarových řešení, kde významně přispívá ke zlepšování uživatelské spokojenosti. Implementace AI funkcionalit do firemního softwaru představuje mnohdy výzvu, která vyžaduje zvážení různých aspektů – od technického řešení přes potřeby zákazníků až po bezpečnost a spolehlivost systému. Následující způsoby využití AI se zaměřují na konkrétní případy v softwaru pro projektový management. Tyto příklady ukazují, jak může být AI technicky nasazena do existujících systémů pro zvýšení produktivity.

3.3.1 Vyhledávání informací

Pro vyhledávání informací je využívána RAG technika pro poskytování přesných a kontextově relevantních odpovědí. Technické řešení je postaveno na vektorové databázi, která ukládá a indexuje dokumenty z různých datových kolekcí. Při dotazu systém nejprve provede vyhledávání relevantních částí dokumentů pomocí sémantického porovnávání, následně tyto informace použije jako kontext pro generování odpovědi. Důležitou potřebou je transparentnost odpovědi. Systém u každé odpovědi uvádí zdroje, ze kterých čerpal informace, což umožňuje uživatelům ověřit správnost a relevanci odpovědi.

Hlavní přínos pro uživatele spočívá v okamžitém přístupu k relevantním informacím bez nutnosti procházet rozsáhlou dokumentací softwaru nebo firemní dokumentací.

3.3.2 Překlad textu

Funkce překladu je implementována pomocí specializovaného promptu, který komunikuje s jazykovým modelem. Systém automaticky detekuje jazyk uživatele z jeho profilového nastavení a zajišťuje překlad do požadovaného jazyka. Technické řešení zahrnuje předzpracování textu, zachování formátování a speciálních značek, a následnou kontrolu kvality překladu.

Pro uživatele tato funkcionalita odstraňuje jazykové bariéry, které jsou nejčastěji v mezinárodních týmech a organizacích. Automatický překlad je dostupný přímo v rozhraní aplikace, což eliminuje potřebu externích překladových nástrojů a udržuje konzistenci terminologie napříč organizací díky datové kolekci.

3.3.3 Oprava gramatiky a pravopisu

Dále častou a oblíbenou funkcí je využití AI pro korekci textu, využívá prompt zaměřený na detekci a opravu gramatických chyb, překlepů a stylistických nedostatků. Technické řešení zahrnuje analýzu textu na několika úrovních. Od základní kontroly pravopisu až po změnu gramatické struktury nebo stylistické koherence. Jazyk dokáže LLM detekovat dle dat vstupující do promptu.

Benefitem pro uživatele je nástroj pro rychlou kontrolu psané komunikace v úkolech a projektech. AI poskytuje zpětnou vazbu v řádu sekund a návrh opravy, což přispívá ke

spolupráci mezi zaměstnanci. Jakmile je uživatel spokojen s úpravou, jedním klikem jí může aplikovat.

3.3.4 Odpověď na tiket

Implementace HelpDesk asistenta využívá sofistikovanou sekvenci promptů pro zpracování zákaznických dotazů. Proces začíná očištěním vstupních dat, následuje vyhledávání podobných případů ve vektorové databázi. Probíhá také analýza úspěšně vyřešených pro generování vhodné odpovědi.

Pro pracovníky podpory představuje asistent významnou časovou úsporu – automaticky navrhuje relevantní odpovědi na základě předchozích zkušeností a dokumentace. Dle interního průzkumu ušetří 5 až 15 minut času na jeden zákaznický požadavek. Pro zákazníky to znamená rychlejší a konzistentnější výstup. Velkou výhodou je, že systém se průběžně učí z dat tiketů a zpětné vazby, čímž se kvalita odpovědí postupně zlepšuje.

3.3.5 Vytváření úkolů

Tato pokročilejší funkcionality využívá sérii promptů pro různé oblasti projektového plánování. Nejprve analyzuje nahraný PDF dokument, extrahuje klíčové informace a následně aplikuje různé prompty pro jednotlivé oblasti – projektové nástěnky, definice úkolů, analýza rizik a další. Každý prompt je optimalizován pro konkrétní část projektového řízení.

Pro uživatele se jedná o automatizovaný nástroj pro vykopnutí projektového plánu, dle předem specifikovaných parametrů a časových údajů v dokumentu. Funkcionality redukuje manuální práci při zakládání projektů a zajišťuje profesionální aplikaci projektových metodik a ověřených projektových postupů.

3.3.6 Sumarizace úkolů

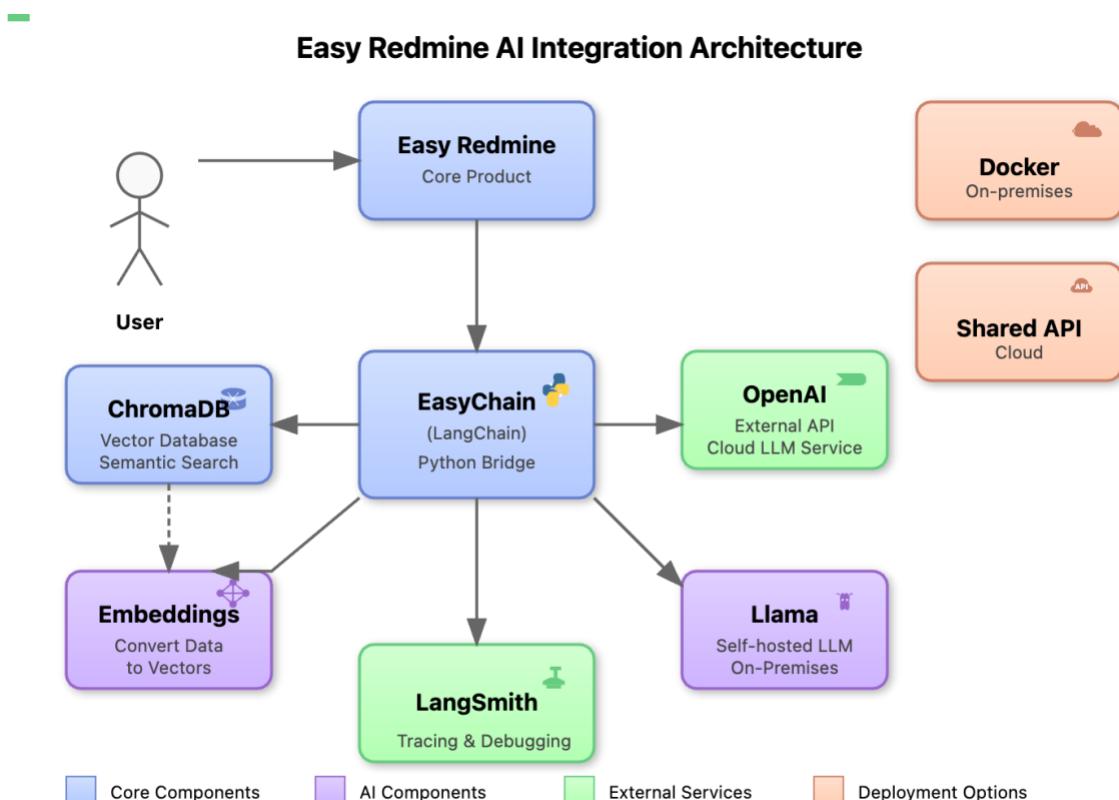
Pro shrnutí úkolů se využívá opět prompt, který analyzuje kontext úkolu a generuje stručný výtah klíčových informací pro uživatele. Technické řešení zahrnuje analýzu historie úkolu, komentářů a souvisejících dokumentů pro vytvoření komplexního, co nejvíce stručného přehledu.

Pro uživatele to znamená možnost rychle pochopit stav a historii úkolu bez nutnosti procházet veškerou komunikaci a dokumentaci. Dle preference uživatele je následně na výběr sumarizace komentářů nebo návrh akčních kroků.

3.4 Architektura AI aplikace Easy Redmine

Jak už bylo zmíněno, základem architektury je aplikace Easy Redmine, platforma pro řízení projektů a správu úkolů. Slouží jako hlavní uživatelské rozhraní a zdroj dat. Pro implementaci AI funkcionalit je využit EasyChain, což je Python kód postavený na

frameworku LangChain, který zprostředkovává komunikaci mezi aplikací a jednotlivými AI komponentami.



Obr. 3.1 Diagram AI architektury Easy Redmine aplikace (vlastní tvorba)

Easy Redmine využívá dva typy jazykových modelů. Cloudové řešení prostřednictvím OpenAI API a firemně hostované open-source modely Llama. Pro cloudové klienty je primárně využíváno OpenAI jako externí cloudová LLM služba, zatímco pro firmy s požadavky na uchování soukromí dat nebo v organizacích s vlastní serverovou infrastrukturou jsou implementovány Llama modely běžící na vlastním hardware.

Dále součástí architektury je ChromaDB, vektorová databáze, kde jsou uloženy firemní data. Ta úzce spolupracuje s komponentou pro tvorbu embeddings, která konvertuje textová data na vektory. Tyto vektory jsou následně ukládány zpět do ChromaDB pro pozdější využití.

Pro účely monitorování uživatelských požadavků se využívá LangSmith, který poskytuje pokročilé nástroje pro debugging a analýzu chování LLM modelů v rámci aplikace. Důležitý je také pro optimalizaci promptů a pochopení rozhodovacích procesů při implementaci AI.

Firma nabízí dvě možnosti nasazení svého software a AI produktů. Docker pro on-premises nasazení a API pro cloudové řešení. Tato flexibilita umožňuje přizpůsobit se specifickým požadavkům klientů, ať už z důvodů regulatorních omezení, bezpečnosti dat nebo s ohledem na geografii.

Design aplikace umožňuje integraci pokročilých LLM funkcionalit do existujícího produktu Easy Redmine při zachování možnosti volby mezi cloudovým a self-hosted řešením.

4 Analýza potřeb firmy pro AI evaluace

Jak již bylo zmíněno, cílem firmy je zjistit, jakým způsobem evaluovat kvalitu promptu a LLM ještě před nasazením změny do produkčního prostředí. Typicky se jedná o proces, kdy vývojář upraví prompt na základě feedbacku od uživatelů a následně je potřeba zjistit, zda nedošlo k nežádoucím změnám.

4.1 Současný stav a výzvy

Proces úpravy promptu nebo nasazení nového modelu je kritickou součástí vývoje AI, neboť změny mohou mít významný dopad na fungování celého softwaru pro klienty. Vývojáři běžně upravují prompty na základě zpětné vazby od uživatelů, avšak doposud neexistuje standardizovaný proces, který by zajistil, že tyto úpravy nevedou k nežádoucím změnám na výstupu.

Evaluace promptů představuje komplexní výzvu, která zahrnuje nejen posouzení kvality generovaných odpovědí, ale také jejich relevanci, přesnost a konzistenci s předchozími výstupy.

Bez systematického přístupu k evaluaci se firma potýká s několika problémy:

1. Subjektivita hodnocení, bez jasně definovaných metrik a procesů je evaluace často založena na subjektivním úsudku vývojářů, což může mít dopad na kvalitu výsledku.
2. Časová náročnost manuálního testování. Ruční kontrola výstupů zdržuje práci vývojářů, zejména pokud se jedná o velké množství případů k otestování. Tento přístup také neumožňuje rychlé iterace a adaptace promptů.
3. Absence měření kvality promptu v čase. Vývojáři nemají okamžitou zpětnou vazbu o dopadu svých změn, což zpomaluje cyklus vývoje a potenciálně umožňuje zhoršení kvality výstupů LLM v produkčním prostředí.

Pro řešení těchto výzev se firma rozhodla implementovat systematický přístup k evaluaci promptů a LLM modelů s využitím platformy LangSmith, která nabízí nástroje pro evaluace, LLM evaluátory, správu datasetů a sledování (tracing) běhu modelů.

4.2 Firemní požadavky na evaluační proces

Na základě současných výzev byly identifikovány následující požadavky firmy na evaluační proces.

Automatizované testování: Cílem je snížit množství manuální práce potřebné pro vyhodnocení kvality promptů. Firma požaduje možnost automatického spouštění testů

z GitLab repozitáře, kde je uložen EasyChain kód. Tato automatizace by měla umožnit rychlé a konzistentní testování změn v promptech.

Definice metrik kvality: Je potřeba definovat a implementovat metriky pro hodnocení kvality generovaných odpovědí. Tyto metriky by měly pokrývat různé aspekty, jako je relevance, přesnost, konzistence a další charakteristiky specifické pro danou funkcionalitu.

Měření kvality v čase: Systém by měl umožňovat sledování vývoje kvality promptů v čase, aby vývojáři mohli identifikovat trendy a potenciální oblasti pro zlepšení.

Integrace s vývojovým prostředím: Evaluační proces musí být integrován do stávajícího vývojové pipeline, konkrétně do GitLab repozitáře, aby se stal přirozenou součástí vývojového cyklu.

Standardizovaný dataset: Pro zajištění konzistentnosti evaluace je třeba vytvořit standardizovaný dataset, který reprezentuje typické případy užití a pokrývají různé scénáře, se kterými se uživatel může setkat.

4.3 Přínosy implementace evaluačního procesu

Implementace systematického evaluačního procesu pro prompty a LLM modely přináší firmě Easy Software výhody v několika oblastech. Tato podkapitola analyzuje konkrétní přínosy s důrazem na kvantifikaci časových, finančních a kvalitativních dopadů.

Časové úspory při vývoji a testování

Současný stav testování promptů je časově náročný proces, který vývojovému týmu zabírá 2-4 hodiny týdně během každého sprintu. Při frekvenci 3-4 úprav promptů měsíčně to představuje časovou investici, která by mohla být využita efektivněji.

Implementovaný evaluační proces tyto časové nároky dramaticky snižuje. Zatímco manuální evaluace vyžaduje několik hodin práce, automatizovaný proces s využitím LLM evaluátorů a LangSmith zabere maximálně 5 minut na jedno spuštění evaluace. To představuje 90% úsporu času při každém testování, což při četnosti úprav promptů v Easy Software znamená přibližně 8-16 ušetřených hodin měsíčně pro vývojový tým.

Tyto ušetřené hodiny mohou být investovány do dalšího vývoje AI funkcionalit nebo optimalizace promptů. Z dlouhodobého hlediska tato časová úspora urychluje celý vývojový cyklus AI produktů a umožňuje firmě rychleji reagovat na zpětnou vazbu od uživatelů.

Prevence proti chybám na klientských aplikacích

Náklady na opravu problémů po nasazení do produkce jsou výrazně vyšší než náklady na jejich prevenci. Ačkoliv je obtížné přesně vyčíslit finanční dopady, časové zpoždění při řešení produkčních incidentů se pohybuje v řádu dnů až týdnů, což má přímý dopad na zákazníky a jejich důvěru v produkt.

Konkrétním příkladem byla situace, kdy úprava promptu pro Knowledge Assistant vedla k neočekávanému chování – jazykový model začal odpovídat všem ve španělštině, ačkoliv v promptu nebylo toto chování specifikováno. Po několika dnech bylo zjištěno, že pokud se v RAG systému objevil webový odkaz se subdoménou „es.“, model interpretoval zdroj jako španělský a generoval výstup v tomto jazyce.

Tento incident demonstruje typický případ, který by systematický evaluační proces zachytil již před nasazením do produkce. Implementovaný systém by při testování identifikoval anomálie v jazykovém výstupu na standardizovaném datasetu, což by umožnilo opravu ještě před nasazením. Tím by se předešlo jak ztrátě důvěry uživatelů, tak i nutnosti řešit kritický problém v produkčním systému.

Finanční dopady

Z finančního hlediska přináší evaluační proces úspory ve třech oblastech. Snížení nákladů na vývoj, např. při hodinové sazbě 1 000 Kč za vývojáře představuje měsíční úspora 8-16 hodin finanční úsporu 8 000 – 16 000 Kč měsíčně, v přepočtu 96 000 Kč až 192 000 Kč ročně.

Redukce nákladů při řešení incidentů. Ačkoliv evaluace nemusí odhalit každou chybu v aplikaci, je těžké přesně kvantifikovat náklady. Nicméně řešení kritického incidentu v produkčním prostředí typicky vyžaduje časové náklady v řádech dnů na odhalení a opravu chyby.

Z opačné strany je nutno započítat i náklady na evaluace, zejména na API volání OpenAI, nicméně jedná se o nízké částky. Jedna evaluace promptu vychází na 4 Kč (0,18 dolarů) za 50 tisíc tokenů při použití GPT-4o modelu.

Také je důležité zmínit, že LangSmith licence se platí již od počátku vývoje AI funkcionalit. Slouží pro sledování (tracing) a odhalení chyb (debugging) klientských aplikací. Z tohoto důvodu není potřeba platit za další licence kvůli implementaci evaluačního procesu, pouze za OpenAI API.

Kvalitativní zlepšení AI řešení

Implementace evaluačního procesu má dopad na zvýšení kvality AI řešení ve firmě. Především přispívá k obnovení důvěry v AI funkcionalitu, která byla v minulosti narušena frekventovanými chybami v produkčním prostředí. Díky systematické evaluaci se zlepšuje konzistence odpovědí, uživatelská zkušenost a standardizuje se kvalita. Proces garantuje minimální úroveň kvality pro všechny AI funkcionality napříč produktovým portfoliem.

Strategické dlouhodobé přínosy

Z dlouhodobého hlediska implementace evaluačního procesu přináší několik výhod. Zaprvé rychlou adaptaci na nově dostupné modely. Evaluační proces umožňuje firmě rychle testovat nové jazykové modely a vyhodnotit jejich výkon na firemních produktech bez zdoluhavého manuálního testování. Tato schopnost je esenciální v rychle se vyvíjející oblasti umělé inteligence, kdy se neustále objevují výkonnější modely.

Dalším přínosem je konkurenční výhoda a informace o výkonu AI pro zákazníka. Schopnost rychle validovat a nasazovat nové AI funkcionality poskytuje firmě konkurenční výhodu v oblasti inovací, což je v IT sektoru nutnost pro zachování relevance.

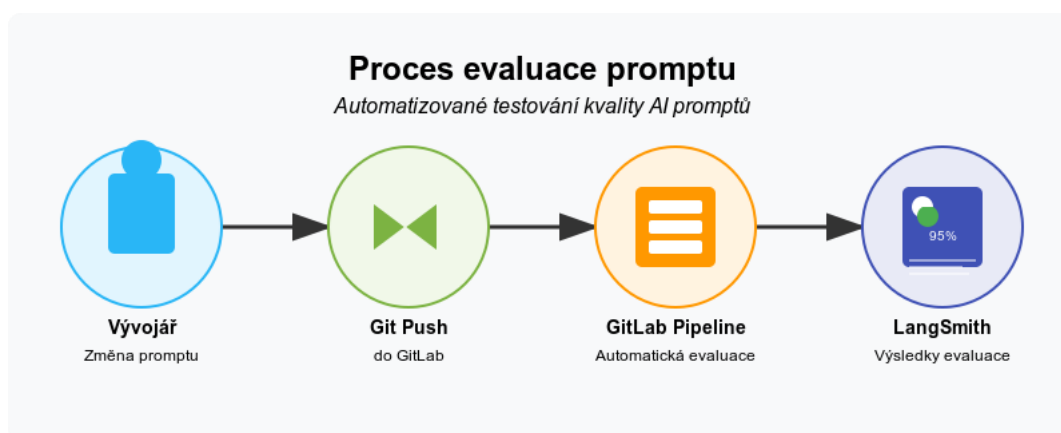
Implementace evaluačního procesu pro prompty a LLM modely představuje pro Easy Software investici s návratností. Kombinace časových úspor, prevence produkčních incidentů, finančních benefitů a kvalitativních zlepšení přináší celkový odhadovaný roční přínos v řádu stovek tisíc korun.

Bonusem jsou i strategické přínosy, které firmě umožňují udržet si konkurenceschopnost v rychle se vyvíjejícím prostředí AI technologií a kontinuálně zlepšovat kvalitu svých produktů. Systematický evaluační proces tak nepředstavuje pouze technické vylepšení, ale stává se nástrojem pro udržitelný rozvoj AI inovací.

5 Implementace evaluační metodiky do firemního prostředí

Tato kapitola se věnuje implementaci evaluačního frameworku v programovacím jazyku Python, zaměřeného na hodnocení promptů a různých LLM modelů s využitím nástroje LangSmith. Proces evaluace je postaven na jednom promptu pro sumarizaci úkolů pro AI Task Summary funkcionalitu. Slouží jako modelový příklad implementace hodnotícího systému.

Autor spolupracuje s vývojovým týmem specializovaný na AI produkty, konkrétně na pozici prompt inženýra. Cílem týmu je integrovat inovativní přístupy projektového řízení a řízení práce za pomoci umělé inteligence. Celý evaluační proces byl navržen autorem, konzultován s vývojáři a produktovým manažerem. V poslední fázi byla automatická evaluace nasazena do procesu.



Obr. 5.1 Grafické znázornění procesu evaluace promptu (vlastní tvorba)

Celý proces funguje následujícím způsobem. Jakmile vývojář nebo prompt inženýr provede úpravu v promptové šabloně, příkazem git push odešle změny do GitLab repozitáře. Tato akce automaticky spustí GitLab pipeline, která zahájí proces automatické evaluace. V rámci pipeline se provádí testování kvality AI promptu pomocí LLM judge evaluátorů nad datasetem a výsledky hodnocení se zaznamenají do LangSmith. Celý tento proces umožňuje automatizované testování kvality promptů, což vývojářům poskytuje okamžitou zpětnou vazbu.

5.1 Metodologie

Před implementací evaluačního systému bylo třeba si ujasnit metodologii, která zajistí objektivní a konzistentní hodnocení kvality LLM výstupů. Popisuje proces návrhu evaluační metodiky, výběr kritérií, základy přístupu LLM-as-judge a metody validace evaluátorů.

5.1.1 Výběr evaluačních kritérií LLM judge

Pro funkcionalitu AI Task Summary bylo potřeba identifikovat aspekty, které uživatelé považují za důležité při práci se shrnutím úkolu. Po analýze zpětné vazby od uživatelů a konzultacích s produktovým týmem byla vybrána čtyři kritéria.

Srozumitelnost (clarity), hlavním účelem shrnutí je poskytnout rychlou a jasnou informaci o úkolu. Zahrnuje logickou strukturu shrnutí a srozumitelnost použitého jazyka. Důvodem zavedení kritéria je, že při nesrozumitelných shrnutích uživatelé raději procházejí celý úkol manuálně.

Věcná správnost (correctness) hodnotí absenci faktických chyb vzhledem k původnímu textu a přesnou reprezentaci klíčových informací. Toto kritérium je zohledněno v kontextu firemního použití, aby výsledné shrnutí úkolů nevedlo k chybným závěrům uživatele.

Stručnost (conciseness), evaluátor hodnotí dodržení požadovaného limitu 200 znaků pro hlavní shrnutí a odrážky ve výstupu. Z dřívějšího uživatelského testování se ukázalo, že kombinace jedné věty a tří odrážek poskytuje vhodnou délku textu.

Informační retence (information retention) byla přidána jako protipól ke stručnosti, aby bylo zajištěno, že snaha o kompaktnost výstupu nevede ke ztrátě kritických informací. Hodnotí zachování informační hodnoty z původního textu.

Kombinace těchto čtyř kritérií vytváří evaluační framework pokrývající jak formální aspekty (stručnost, struktura), tak obsahové aspekty (správnost, úplnost) generovaných shrnutí.

5.2 Architektura evaluačního systému

Architektura evaluačního systému je navržena s důrazem na modularitu, znovupoužitelnost a snadnou integraci do existujících vývojových procesů. Základní komponenty systému jsou uloženy v GitLab repozitáři „EASY_AI_CHAIN“, který obsahuje veškeré potřebné zdrojové kódy a konfigurační soubory.

Výpis 5.1 GitLab struktura repozitáře pro evaluace

```
EASY_AI_CHAIN/
├── data/
├── doc/
├── doc-internal/
├── evals/
├── prompt_templates/
├── src/
│   ├── debug_scripts/
│   ├── easy_pipelines/
│   ├── embeddings/
│   ├── evals/
│   │   └── read_assistant/
│   │       └── evaluators/
│   │           ├── score_evaluation.py
│   │           ├── summarize_clarity_evaluator.py
│   │           └── summarize_conciseness_evaluator.py
```

```

├── summarize_correctness_evaluator.py
├── summarize_info_retention_evaluator.py
├── prompts/
│   └── summarize_eval_prompt.md

```

V rámci adresářové struktury repozitáře je evaluační framework logicky rozdělen do několika částí:

1. **LLM evaluátoři:** Adresář „evals/read_assistant/evaluators/“ obsahuje jednotlivé evaluační moduly (prompty) implementované jako samostatné Python soubory. Každý evaluátor je specializován na hodnocení konkrétního aspektu generovaných odpovědí, jako je srozumitelnost, věcná správnost, stručnost a informační hodnota. Tato modularita umožňuje snadno přidávat nové evaluátory nebo upravovat stávající podle měnících se požadavků.
2. **Šablony promptů:** Adresář „evals/read_assistant/prompts/“ obsahuje šablony promptů ve formátu Markdown, které jsou testovány v rámci evaluačního procesu. Oddělení promptů od implementační logiky umožňuje i dalším členům týmu (například prompt inženýrům) snadno upravovat textové šablony bez nutnosti zasahovat do kódu.
3. **Evaluační orchestrátor** – Hlavní skript summarization_evaluator.py se nachází v adresáři „evals/read_assistant/“ a slouží jako orchestrátor celého evaluačního procesu. Tento skript zajišťuje načtení promptů, inicializaci jazykových modelů, spuštění evaluátorů a agregaci výsledků.

Tato architektura umožňuje flexibilní konfiguraci evaluace pro různé typy asistentů a různá hodnotící kritéria při zachování jednotného přístupu k hodnocení napříč produktovým portfoliem.

└── summarization_evaluator.py

5.3 Příprava dat a evaluačního datasetu

Dalším nezbytným krokem bylo vytvoření datasetu, na kterém se budou pouštět evaluace. Dataset byl navržen s ohledem na pokrytí široké škály scénářů, se kterými se uživatelé setkávají. Vzhledem k definovaným požadavkům byl dataset vytvořen jako kombinace reálných a syntetických dat, která věrně simulují strukturu a obsah úkolů.

Struktura každého případu (example) v datasetu obsahuje informace typické pro úkoly v aplikaci Easy Redmine: název úkolu, zadání úkolu, data o úkolu a komentáře. Pro zachování konzistence a současně zajištění diverzity byly při tvorbě datasetu aplikovány následující parametry:

- **Délka kontextu:** Dataset záměrně obsahuje tři krátké úkoly s minimem detailů (100-200 znaků) a tři rozsáhlé úkoly s dlouhou historií komentářů (přes 3 000 znaků). Tato variabilita umožňuje testovat schopnost modelu pracovat s dlouhým a krátkým kontextem. Obě varianty jsou rozděleny do kategorií pod názvy short_context a long_context.

- **Různorodá struktura:** V datasetu jsou zahrnuty úkoly s různou úrovní složitosti od jednoduchých zadání (např. je zapotřebí aktualizovat produktovou dokumentaci) až po komplexní úkoly se seznamy, odkazy, formátovaným textem a speciálními znaky. Díky tomu se testuje robustnost sumarizačního LLM při zpracování různě strukturovaných dat, označeno kategorií `formatting_challenge`.
- **Extrémní případy:** Existuje několik netypických případů, jako jsou úkoly s minimálním množstvím informací nebo naopak velmi detailní popisy s množstvím redundantních informací. Tyto záznamy pomáhají identifikovat potenciálně slabá místa ve výsledcích v rámci kategorie `edge_case`.
- **Jazyková rozmanitost:** Součástí datasetu jsou úkoly v různých jazycích (angličtině, francouzštině nebo čínštině), které testují schopnost modelu správně detekovat jazyk vstupního textu a generovat odpověď ve stejném jazyce, kategorie `language_variation`. Firma Easy Software má klienty po celém světě, včetně asijských zemí, proto je zvolena jazyková diverzita.

Pro vytvoření datasetu byly nejprve shromážděny anonymizované úkoly z interních projektů. Ty byly následně analyzovány a doplněny o syntetické příklady vytvořené podle kritérií. Důvodem pro doplnění reálných dat syntetickými byla zejména obtížnost nalezení dostatečně diverzní sady příkladů splňujících všechna definovaná kritéria v původních datech, jako jsou extrémní příklady, multijazyčnost, multiplikace příkladů.

Výsledný evaluační dataset (příloha A) obsahuje 15 vytvořených příkladů, které pokrývají všechny identifikované případy užití. Tento dataset je uložen v LangSmith, kde je dostupný pro opakované spouštění evaluací a sledování vývoje kvality v čase. Díky konzistentnímu datasetu je možné objektivně porovnávat různé verze promptů nebo výkon různých jazykových modelů na stejné sadě příkladů.

5.4 Implementace evaluátorů pro sumarizační prompt

Pro evaluaci promptu sumarizaci úkolů (funkcionalita AI Task Summary), byly implementovány čtyři specializovaní evaluátoři, kteří posuzují různá kritéria generovaných shrnutí. Každý evaluátor je reprezentován v kódu samostatný Python objekt, který implementuje specifické hodnotící kritérium a využívá LLM jako hodnotitele kvality výstupů.

5.4.1 Score Evaluation

Základní stavební kámen evaluačního systému tvoří skript `score_evaluation.py` (příloha B), který definuje standardizovaný formát hodnocení prostřednictvím Pydantic knihovny.

Výpis 5.2 Ukázka score_evaluation.py skriptu

```
class ScoreEvaluation(BaseModel):
    score: float = Field(
        description="Score from 0 to 1 where 0 is the worst and 1 is the best. A score of 1 means "
        "that the response perfectly fits the criteria, "
        "whereas a score of 0 means that "
        "it did not fit the criteria at all."
    )
    reasoning: str = Field(
        description="Reasoning behind the score"
    )
```

Třída `ScoreEvaluation` definovaná v tomto modulu obsahuje dvě vlastnosti. `Score` pro numerické hodnocení v rozsahu 0-1, kde 1 představuje stoprocentní splnění daného kritéria a 0 nesplnění kritéria. Vlastnost `reasoning` je textové odůvodnění, které popisuje, proč bylo dané skóre přiděleno.

Tento jednotný formát pomocí Pydantic knihovny zajišťuje, že všechny evaluátory poskytují výsledky ve stejné struktuře, což významně usnadňuje jejich agregaci a následnou analýzu. Zároveň kombinace kvantitativního skóre a kvalitativního odůvodnění poskytuje detailní vhled do hodnocení než samostatná číselná hodnota. Příklad výstupu je viditelný na obrázku (obr. 2.2).

5.4.2 Evaluátor pro posouzení srozumitelnosti (clarity)

Evaluátor pro hodnocení srozumitelnosti (`summarize_clarity_evaluator.py`, příloha C) posuzuje, jak je generované shrnutí jasné a kvalitně strukturované. Funkce `clarity_evaluator` nejprve vypíše informaci o zpracovávaných datech a následně využívá OpenAI API pro získání hodnocení. Pro komunikaci s OpenAI API se využívá `openai_client` z balíčku `langsmith`. Důvodem je kompatibilita tracingu pro sledování všech API volání.

Výpis 5.3 Ukázka summarize_clarity_evaluator.py skriptu

```
openai_client = wrappers.wrap_openai(OpenAI())

def clarity_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí srozumitelnost a jasnost shrnutí."""
    print(f"clarity_evaluator OUTPUTS: {outputs.keys()}")
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing the clarity of summaries."},
            {"role": "user", "content": f"""
            Evaluate the CLARITY of the following summary.

            A score of 1 is best and means that the response fits the criteria, whereas a score of 0 is worst and means that it did not fit the criteria.

            Consider the following when evaluating clarity:
```

```

        - Is the summary well-structured and easy to understand?
(higher score)
        - Is the language clear and precise? (higher score)
        - Are there any confusing elements or ambiguous statements?
(lower score)
        - Does the summary follow a logical structure? (higher score)

    Original text:
    {inputs["text"]}

    Summary:
    {outputs["output"]}
    """
],
    response_format=ScoreEvaluation,
)
return {"score": response.choices[0].message.parsed.score,
        "reasoning": response.choices[0].message.parsed.reasoning}

```

Tento evaluátor, jako všechny následující v kapitole, má definovanou systémovou roli (system) a uživatelský vstup (user) s instrukcemi. Úkolem je analyzovat strukturu vygenerované odpovědi od LLM, dále srozumitelnost jazyka, výskyt nejasných tvrzení a celkovou logickou strukturu. Využívá model GPT-4o jako objektivního hodnotitele, kterému je předložen originální text a vygenerované shrnutí.

Pro zajištění konzistentního formátu odpovědi je použita zmíněná ScoreEvaluation třída. Výsledné hodnocení a zdůvodnění je následně vráceno ve formátu JSON, který je kompatibilní s LangSmith pro vizualizaci výsledků přímo v nástroji.

5.4.3 Evaluátor pro posouzení věcné správnosti (correctness)

Další evaluátor kontroluje věcnou správnost (summarize_correctness_evaluator.py, příloha D). Jeho úkolem je ověřit, zda shrnutí věrně reprezentuje informace obsažené v původním textu, bez přidávání, vynechávání nebo zkreslování faktů. Struktura kódu je stejná, jako u předchozího evaluátora, opět používá stejné importované knihovny, role promptu a formát výstupu.

Výpis 5.4 Ukázka summarize_correctness_evaluator.py skriptu

```

# Evaluátor pro Correctness
def correctness_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí správnost shrnutí vzhledem k původnímu textu."""
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing the
correctness of summaries."},
            {"role": "user", "content": f"""
Evaluate the CORRECTNESS of the following summary relative to
the original text.

A score of 1 is best and means that the response fits the
criteria, whereas a score of 0 is worst and means that it did not fit the
criteria.
"""
}
]
    )
    return {"score": response.choices[0].message.parsed.score,
            "reasoning": response.choices[0].message.parsed.reasoning}

```

```

        Consider the following when evaluating correctness:
        - Does the summary contain factual errors or misrepresentations?
        (lower score)
        - Does the summary accurately represent the key information
        from the original text? (higher score)
        - Are there any inaccuracies or omissions that change the
        meaning? (lower score)

        Original text:
        {inputs["text"]}

        Summary:
        {outputs["output"]}
        """
    ],
    response_format=ScoreEvaluation,
)
return {"score": response.choices[0].message.parsed.score,
        "reasoning": response.choices[0].message.parsed.reasoning}

```

Instrukce evaluátora se zaměřují na faktickou přesnost, detekci případných chyb nebo dezinterpretací v shrnutích. Posuzuje, zda odpověď reprezentuje klíčové informace z původního textu (úkolů) a zda neobsahuje nepřesnosti, které by měnily význam.

Nízké skóre v této metrice může indikovat potenciální problém v prompt engineeringu nebo chybu v RAG systému.

5.4.4 Evaluátor pro posouzení stručnosti odpovědi (conciseness)

Evaluátor pro kritérium stručných odpovědí (summarize_conciseness_evaluator.py, příloha E) ohodnocuje, zda dodržuje předepsaný formát a odpovědi jsou výstižné bez přebytečných detailů. Příliš dlouhé shrnutí může uživatele vyčerpávat a příliš stručná odpověď nemusí vystihnout podstatu úkolů.

Výpis 5.5 Ukázka summarize_conciseness_evaluator.py skriptu

```

# Evaluátor pro Conciseness
def conciseness_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí stručnost a výstižnost shrnutí."""
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing the
conciseness of summaries."},
            {"role": "user", "content": f"""
Evaluate the CONCISENESS of the following summary.

A score of 1 is best and means that the response fits the
criteria, whereas a score of 0 is worst and means that it did not fit the
criteria.

Check specifically:
- Summary is limited to 200 characters as required
- Has exactly THREE bullet points under Key points

```

```

        - No excessive information
        - Includes only essential information without unnecessary
details

    Original text:
    {inputs["text"]}

    Summary:
    {outputs["output"]}
    """
],
    response_format=ScoreEvaluation,
)
return {"score": response.choices[0].message.parsed.score,
        "reasoning": response.choices[0].message.parsed.reasoning}

```

Na rozdíl od předchozích evaluátorů, které hodnotí spíše kvalitativní aspekty, tento evaluátor ověřuje dodržování konkrétních formálních požadavků. Kontroluje, zda shrnutí splňuje limit 200 znaků pro odpověď a přesný počet odrážek v seznamu. Z testování sumarizačního promptu se zjistilo, že LLM může vygenerovat mnohem více odrážek, než je specifikováno v promptu. Proto je zavedena kontrola i na tuto limitaci výstupu.

5.4.5 Evaluátor pro posouzení informační retence odpovědi (information retention)

Úkoly typicky obsahují velké množství komentářů a důležité informace, proto byl navržen evaluátor (summarize_info_retention_evaluator.py, příloha F) pro hodnocení informační retence. Zatímco věcná správnost kontroluje absenci chyb, informační retence hodnotí úplnost zmíněných informací.

Výpis 5.6 Ukázka summarize_info_retention_evaluator.py skriptu

```

# Evaluátor pro Information Retention
def info_retention_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí, jak dobře jsou zachovány faktické informace v shrnutí."""
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing
information retention in summaries."},
            {"role": "user", "content": f"""
Evaluate the INFORMATION RETENTION of the following summary.

A score of 1 is best and means that the response fits the
criteria, whereas a score of 0 is worst and means that it did not fit the
criteria.

Consider the following when evaluating information retention:
- Does the summary retain all critical information from the
original text? (higher score)
- Are important details or key points missing? (lower score)
- Does the summary maintain the core message and essential
context? (higher score)

```

```

        Original text:
        {inputs["text"]}

        Summary:
        {outputs["output"]}
        """
    ],
    response_format=ScoreEvaluation,
)
return {"score": response.choices[0].message.parsed.score,
        "reasoning": response.choices[0].message.parsed.reasoning}

```

Tato metrika je obzvláště důležitá pro AI Task Summary, kde je cílem poskytnout uživateli rychlý, ale hlavně kompletní přehled o úkolu bez nutnosti procházet celou historii. Evaluátor má definováno hodnotit v promptu, zda jsou zachovány všechny důležité informace, zda nechybí zmínit detaily úkolu a zda shrnutí zachovává hlavní sdělení a kontext.

5.5 Evaluační prompt pro AI Task Summary

Nedílnou součástí evaluačního procesu je samostatný prompt, který je příčinou hodnocení. Pro funkcionalitu byl vložen prompt v Markdown (summarize_eval_prompt.md), jedná se o vhodný formát pro komunikaci s LLM.

V úvodní části definuje roli modelu jako generátor shrnutí. Prostřední sekce obsahuje konkrétní instrukce pro formát výstupu – shrnutí v jedné větě omezené na 200 znaků a přesně tři odrážky. Sekce s instrukcemi dále specifikuje, že výstup má být vždy generován ve formátu Markdown a v jazyce shodném s jazykem vstupního textu. Pokud by byl definován jiný formát promptu, uživateli by se výstup nemusel zobrazit správně.

Výpis 5.7 Ukázka OpenAI šablony promptu

```

# INTRODUCTION
You are an executive summary generator.

# TASK
Your task is to summarize text enclosed in triple quotes.
1. Generate a short summary of the task in 1 sentence (limit 200
characters). Start with the H2 title "Summary".
2. Generate exactly THREE bullet points to explain the rest of the text.
Start with the H2 title "Key points". Do not exceed three bullet points.

# INSTRUCTIONS
ALWAYS generate output in markdown.
TARGET LANGUAGE: {language_detected}

CONTEXT:
"""{text}"""

```

Pro zajištění kompatibility s různými LLM modely, zejména pro serverové nasazení aplikace s modelem Llama, byl implementován i alternativní formát promptu s použitím specifických značek.

Výpis 5.8 Ukázka Llama šablony promptu

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are an executive summary generator.
<|eot_id|><|start_header_id|>user<|end_header_id|>

# TASK
Your task is to summarize text enclosed in triple quotes.
1. Generate a short summary of the task in 1 sentence (limit 200
characters). Start with the H2 title "Summary".
2. Generate exactly THREE bullet points to explain the rest of the text.
Start with the H2 title "Key points". Do not exceed three bullet points.

# INSTRUCTIONS
ALWAYS generate output in markdown.
TARGET LANGUAGE: {language_detected}

CONTEXT:
""""{text}""""
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Speciální značky, tzv. tokeny (tokens), jsou součástí formátu používaného v rámci Llama LLM. Tyto tokeny slouží k přesnému strukturování konverzace a zajištění správné interpretace jednotlivých částí promptu jazykovým modelem. Každá značka má svůj účel v rámci komunikačního protokolu. Zde jsou nejdůležitější:

- **<|begin_of_text|>** - Označuje začátek celého konverzačního kontextu. Tento token signalizuje Llama modelu, že následuje strukturovaná konverzace, která vyžaduje specifické zpracování. Speciální token nemá párovou (koncovou) značku.
- **<|start_header_id|>** a **<|end_header_id|>** - Tyto párové značky ohraničují identifikátor role v konverzaci. Prostor mezi těmito značkami obsahuje buď „system“, „user“ nebo „assistant“, čímž je jasně definováno, kdo je autorem následujícího textu neboli promptu. Bez těchto informací mohou mít modely problémy s rozlišením, které části textu jsou instrukce a které jsou očekávaným výstupem.
- **<|eot_id|>** - Signalizuje přechod k dalšímu účastníkovi konverzace a konec promptu. Tento token pomáhá modelu správně identifikovat hranice mezi jednotlivými vstupy.

Implementace těchto značek vychází z oficiální dokumentace a doporučení pro modely Llama, které mají specifický formát chatovacího protokolu. Zatímco modely od OpenAI (jako GPT-4) nevyžadují další modifikace, modely Llama vyžadují explicitní označení pomocí těchto speciálních tokenů (*Llama 3.1 Documentation*, b.r.).

Použití této specifické syntaxe je nezbytné pro správné fungování Llama modelů a ilustruje potřebu přizpůsobit prompty specifikům jednotlivých modelů, což je přesně případ firmy Easy Software s její podporou jak cloudových, tak serverových řešení.

5.6 Orchestrace evaluačního procesu

Celý evaluační proces je koordinován prostřednictvím hlavního skriptu `summarization_evaluator.py` (příloha G), který zajišťuje následující funkce:

1. Načtení šablony promptu.
2. Výběr jazykového modelu pro generování odpovědi.
3. Zpracování vstupních dat a generování LLM výstupu.
4. Hodnocení evaluace prostřednictvím jednotlivých LLM judge evaluátorů.
5. Výsledky evaluací a agregace výsledků evaluace.

5.6.1 Import knihoven

V úvodní části skriptu jsou importovány všechny potřebné knihovny a moduly.

Výpis 5.9 Ukázka importu knihoven `summarization_evaluator.py` kódu

```
import argparse
import os
from pathlib import Path

from dotenv import load_dotenv
from fastapi import FastAPI
from langsmith import Client

import settings as server_settings
from app_state import app_state
from chain_api_client import ChainApiClient
from create_app import create_app
from evals.read_assistant.evaluators.summarize_clarity_evaluator import clarity_evaluator
from evals.read_assistant.evaluators.summarize_conciseness_evaluator import conciseness_evaluator
from evals.read_assistant.evaluators.summarize_correctness_evaluator import correctness_evaluator
from evals.read_assistant.evaluators.summarize_info_retention_evaluator import \
    info_retention_evaluator
from routers.api import ApiGenerateRequest
from utils import sign_jwt
```

Knihovna `argparse` umožňuje zpracování argumentů příkazové řádky, což je využito pro specifikaci modelu při spuštění evaluace. Modul `pathlib` poskytuje objektově orientovaný přístup k souborovému systému a je využit pro práci s cestami k souborům. Knihovna `dotenv` slouží k načítání proměnných prostředí ze souboru „.env“, což je bezpečný způsob správy konfiguračních proměnných a API klíčů.

Framework `FastAPI` je využit pro vytvoření lokálního API rozhraní, které simuluje produkční prostředí aplikace `Easy Redmine`. Klient `langsmith` zajišťuje integraci s platformou `LangSmith` pro sledování a vyhodnocování evaluací.

Dále jsou importovány vlastní moduly aplikace – `app_state` pro správu stavu aplikace, `chain_api_client` pro komunikaci s API v rámci EasyChain, a všechny čtyři popsané evaluátory.

5.6.2 Konfigurace a inicializace

Po importu knihoven následuje inicializace konfigurace a proměnných prostředí.

Výpis 5.10 Ukázka konfigurace API v `summarization_evaluator.py` kódu

```
load_dotenv()

# Přístup k API klíčům
langsmith_api_key = os.getenv("LANGSMITH_API_KEY")
openai_api_key = os.getenv("OPENAI_API_KEY")
langsmith_tracing = os.getenv("LANGSMITH_TRACING")
```

Funkce `load_dotenv()` načítá proměnné, obsažené v souboru API klíče. Tento přístup zajišťuje, že citlivé údaje nejsou uloženy přímo v kódu. Následně jsou tyto proměnné načteny do proměnných pro další použití.

Dále je definována pomocná funkce `_read_prompt()`, která načte prompt šablonu z disku.

Výpis 5.11 Ukázka načítání promptu v `summarization_evaluator.py` kódu

```
def _read_prompt() -> str:
    """
    Vytvoří prompt pro generování shrnutí podle zadané struktury.
    """
    current_folder = Path(__file__).parent
    prompts_folder = current_folder / "prompts"
    with open(prompts_folder / "summarize_eval_prompt.md", "r",
encoding="utf-8") as f:
        content = f.read()
    return content
```

5.6.3 Spuštění evaluace

Metoda `run` zajišťuje inicializaci prostředí, výběr evaluačního datasetu a spuštění samotné evaluace.

Výpis 5.12 Ukázka funkce `run` v `summarization_evaluator.py` kódu

```
def run(self):
    server_settings.JWT_SECRET_KEY = "JWT_SECRET_KEY"
    server_settings.JWT_ALGORITHM = "HS256"
    token = sign_jwt("test_user")

    self.app = create_app(app_state=app_state, unit_test=True)

    self.chain_api = ChainApiClient(app=self.app, jwt_token=token)

    llm_registry = app_state.llm_registry
```

```

provider = llm_registry.get_provider(llm_id=self.llm_id)
model_name = provider.model_name

metadata = {
    "llm_id": self.llm_id,
    "model_name": model_name,
    "evaluator_llm_id": "gpt-4o",
    "evaluator_model_name": "gpt-4o",
}

# Fetch the existing dataset
all_datasets = self.ls_client.list_datasets()
all_datasets = list(all_datasets)
dataset_name = "Easy AI Read Assistant - Summarize 2.0"
my_datasets = list(filter(lambda ds: ds.name == dataset_name,
all_datasets))
if len(my_datasets) == 0:
    raise ValueError(f"No datasets {dataset_name} found")
if len(my_datasets) != 1:
    raise ValueError(f"Multiple datasets with name {dataset_name}")
dataset = my_datasets[0]
self.ls_client.read_dataset(dataset_id=dataset.id)
print(f"Using dataset '{dataset.name}' with ID: {dataset.id}")

print(f"Starting evaluation for dataset {dataset.name}...")

experiment_results = self.ls_client.evaluate(
    self.process_example,
    data=dataset.id,
    evaluators=[
        correctness_evaluator,
        conciseness_evaluator,
        info_retention_evaluator,
        clarity_evaluator
    ],
    experiment_prefix="summary_prompt_evaluation",
    max_concurrency=2,
    metadata=metadata
)

print(f"Results: {experiment_results}")

```

Tato metoda nejprve inicializuje aplikační prostředí – nastavuje JWT (JSON Web Token) konfiguraci pro autentizaci, vytváří FastAPI aplikaci a inicializuje klienta pro komunikaci s API. Následně získává informace o požadovaném modelu z registru modelů a vytváří metadata pro evaluační experiment.

Dále metoda vyhledává evaluační dataset v LangSmith podle identifikátoru. Pokud je vše v pořádku, spouští evaluaci pomocí metody evaluate klienta LangSmith.

V kódu v rámci spuštění evaluace jsou specifikovány zmíněné čtyři evaluátoři, budou použity pro hodnocení generovaných odpovědí, prefix experimentu pro lepší organizaci v rozhraní a maximální počet souběžných evaluací pro optimalizaci výkonu.

Poslední část skriptu obsahuje kód, který se provede pouze při spuštění souboru jako samostatného programu.

Výpis 5.13 Ukázka spuštění evaluace v summarization_evaluator.py kódu

```
if __name__ == "__main__":  
    arg_parser = argparse.ArgumentParser()  
    arg_parser.add_argument("--llm-id", type=str, required=False,  
default="gpt-4o")  
    args = arg_parser.parse_args()  
  
    evaluator = SummarizationEvaluator(llm_id=args.llm_id)  
    evaluator.run()
```

Zde je vytvořen parser argumentů příkazové řádky, který umožňuje specifikovat identifikátor jazykového modelu pomocí přepínače --llm-id. Výchozí hodnota je nastavena na gpt-4o, což je aktuálně vhodný model z hlediska cenové dostupnosti a výkonu od OpenAI. Následně je vytvořena instance třídy SummarizationEvaluator s poskytnutým identifikátorem modelu a spuštěna metoda run().

Díky této implementaci může vývojář jednoduše otestovat evaluaci pomocí příkazu.

```
python summarization_evaluator.py --llm-id gpt-4o
```

Evaluační systém následně vyhodnotí kvalitu shrnutí generovaných zadaným modelem podle předem definovaných kritérií a výsledky se zobrazí ve webovém rozhraní LangSmith.

6 Validace výsledků implementace evaluačního konceptu

Výsledky implementovaného evaluačního systému jsou znázorněny v této kapitole. Konkrétně se jedná o ověření spolehlivosti a konzistence vytvořených evaluátorů při posuzování kvality promptů a LLM. Cílem validace bylo zjistit, zda je navržený evaluační systém vhodný pro produkční nasazení a poskytuje konzistentní výsledky při opakovaném testování.

6.1 Metodika

Pro ověření spolehlivosti evaluačního konceptu byl zvolen přístup opakovaného testování stejného nebo modifikovaného promptu na identickém datasetu. Konkrétně bylo puštěno 40 evaluačních testů na datasetu, který je rozdělen do pěti testovacích kategorií. Polovina testů byla provedena s GPT-4o modelem a u druhé poloviny byl zvolen Llama-3.3-70B-Instruct model. Cílem je ověřit konzistenci výsledků při různých podmínkách, jako jsou různé scénáře, modely a jak budou reagovat na změny evaluátorů.

- Experiment #1: Testování originálního promptu bez jakýchkoliv úprav, což sloužilo jako referenční hodnota pro ostatní experimenty.
- Experiment #2: Testování modifikovaného promptu, z něhož byla záměrně odstraněna klíčová proměnná {text}, která obsahuje data z datasetu.
- Experiment #3: Testování promptu bez instrukcí ohledně očekávané délky výstupu (odstranění limitu 200 znaků).
- Experiment #4: Testování promptu bez instrukcí týkajících se struktury výstupu (odstranění formátovacích pokynů).

Všechny experimenty byly provedeny na standardizovaném evaluačním datasetu obsahujícím 15 příkladů, které pokrývají různé případy použití. Hodnocení bylo prováděno prostřednictvím čtyř implementovaných evaluátorů (clarity, conciseness, correctness a information retention), přičemž každý evaluátor poskytoval skóre v rozsahu 0-1 společně s textovým odůvodněním.

Pro vyhodnocení výsledků experimentů byl použit následující postup:

1. Agregace hodnocení napříč všemi příklady v datasetu pro každý evaluátor a výpočet průměrného skóre v procentech pro každou metriku.
2. Porovnání výsledků mezi jednotlivými experimenty a modely, se zaměřením na odchylky v hodnocení způsobené konkrétními modifikacemi promptu.
3. Ověření konzistence evaluačního systému pomocí opakovaného spouštění stejných testů a výpočtu odchylek mezi jednotlivými běhy.

4. Kvantitativní a kvalitativní vyhodnocení rozdílů ve výkonu mezi modely, včetně jejich reakce na různé typy modifikací v promptech.

Tato metodika umožňuje nejen kvantitativně měřit dopad změn v promptech na kvalitu generovaných odpovědí, ale také ověřit, zda je evaluační framework dostatečně stabilní a spolehlivý pro implementaci.

6.2 Výsledky automatických evaluací

Výsledky experimentů zachycené v LangSmith poskytly komplexní vhled na výkon evaluačního systému a potvrdily jeho schopnost konzistentně identifikovat problémy v promptech. Došlo k zajímavému zjištění ohledně rozdílů mezi použitými modely a prompty.

6.2.1 Srovnání experimentů a modelů

Tabulka 6.1 Srovnání jednotlivých experimentů (#1, #2, #3, #4) a modelů (GPT-4o, Llama 3.3)

Experiment	LLM	Clarity (%)	Conciseness (%)	Correctness (%)	Info_retention (%)
#1	GPT-4o	92	92	94	89
	Llama 3.3	91	91	85	75
#2	GPT-4o	17	11	1	1
	Llama 3.3	31	41	5	6
#3	GPT-4o	96	81	99	92
	Llama 3.3	95	86	96	87
#4	GPT-4o	99	38	99	95
	Llama 3.3	99	45	98	94

Tabulka zobrazuje výsledky jednotlivých evaluačních experimentů v procentech. Hodnoty vyjadřují kvalitu výsledku dle použitého promptu a modelu. V řádcích jsou čtyři různé varianty promptů, každá měřená pro dva různé modely (sloupec LLM) a ve sloupcích jsou čtyři hodnotící kritéria (srozumitelnost, stručnost, správnost a informační retence).

Celkem tabulka znázorňuje, jak jednotlivé modely skórovaly v různých kritériích při různých experimentech. Čím vyšší je procento, tím lepší je výsledek. Modrou barvou jsou rozlišeny sloupce, které mají společnou metriku pro porovnání modelů.

Experiment #1

V rámci prvního experimentu originální prompt dostal nejvyšší a nejvyváženější skóre napříč všemi metrikami, což potvrzuje jeho kvalitu. Při srovnání modelů vykazovaly GPT a Llama podobné výsledky v metrikách srozumitelnosti (92 % vs. 91 %) a stručnosti (92 %

vs. 91 %), avšak GPT výrazně převyšoval Llama model v metrikách správnosti (+9 procentních bodů) a informační retence (+14 procentních bodů).

Experiment #2

Odstranění kontextu v druhém experimentu vedlo k dramatickému propadu všech hodnocených metrik u obou modelů kvůli chybným výsledkům. Skóre se u metriky správnosti a informační retence propadlo téměř na nulu (1 % a 1 % u GPT), což jednoznačně ukazuje, že evaluační systém dokáže spolehlivě identifikovat zásadní chyby v promptu. Zajímavé je, že v tomto experimentu Llama model paradoxně dosáhl vyššího skóre u stručnosti (41 % oproti 11 % u GPT), což lze vysvětlit tím, že model vracel odpověď, kterou evaluátor vyhodnotil jako relativně stručnou.

Experiment #3

Odstranění limitu 200 znaků ve třetím experimentu vedlo k mírnému poklesu hodnocení stručnosti (81 % u GPT a 86 % u Llama), zatímco ostatní metriky zůstaly na vysoké úrovni, jako v případě prvního experimentu. Tento výsledek měl potvrdit citlivost evaluátorů na délku výstupu a jeho schopnost penalizovat příliš dlouhé odpovědi. Nicméně pokles nebyl značný.

Experiment #4

Ve čtvrtém experimentu, kdy byly odstraněny instrukce týkající se struktury výstupu, došlo k nejvýraznějšímu propadu u metriky stručnosti (38 % u GPT a 45 % u Llama). To demonstruje důležitost instrukcí pro formátování výstupu a zároveň schopnost evaluačního systému na tuto oblast spolehlivě reagovat. Naproti tomu metriky srozumitelnosti a správnosti zůstaly na téměř perfektní úrovni, což naznačuje, že absence strukturálních instrukcí ovlivňuje především formální charakteristiky výstupu, nikoliv jeho obsahovou kvalitu.

6.2.2 Testování konzistence a spolehlivost hodnocení

Tabulka 6.2 Srovnání jednotlivých opakovaných experimentů (#1, #2, #3, #4) a modelů (GPT-4o, Llama 3.3)

Experiment	LLM	Clarity (%)	Conciseness (%)	Correctness (%)	Info_retention (%)
#1	GPT-4o	95	87	93	89
	Llama 3.3	93	91	87	78
#2	GPT-4o	14	18	0	4
	Llama 3.3	28	33	1	9
#3	GPT-4o	97	83	97	89
	Llama 3.3	95	81	93	86
#4	GPT-4o	98	41	99	96
	Llama 3.3	99	44	98	93

Pro ověření konzistence a spolehlivosti evaluačního systému byly totožné experimenty opakovány s identickými parametry. Pro validaci výsledků byly puštěny dva po sobě jdoucí běhy testů pro přesnější výsledky. Struktura tabulky zůstává stejná, akorát jsou zde zaznamenány nové hodnoty z experimentů.

Opakované spuštění evaluací potvrdilo konzistenci hodnotícího systému. Rozdíl mezi první a druhou tabulkou je minimální, žádná metrika se neodchýlila o více než $\pm 0,08$ procentního bodu. Směrodatné odchylky u jednotlivých metrik napříč modely byly rovněž nízké, konkrétně $\leq 0,026$ pro srozumitelnost a věcnou správnost. Informační retence měla nejvyšší rozptyl $\leq 0,05$ procentního bodu. V kontextu absolutní změny je rozptyl na nízkých hodnotách. Směrodatná odchylka byla vypočítána pomocí SMODCH.VÝBĚR.S funkce v Microsoft Excelu.

Míra konzistence poskytuje důkaz o spolehlivosti evaluačního frameworku, že systém bude vykazovat stabilní výsledky při úpravách promptu nebo změnách jazykového modelu, jelikož se odráží na kvalitě evaluací.

6.3 Validace LLM evaluátorů pomocí lidského hodnocení

Tabulka 6.3 Kvalitativní analýza slovního ohodnocení evaluátorů na reprezentativním vzorku

Experiment	Model	Metrika	Skóre	Slovní shrnutí
#1	GPT-4o	Conciseness	90 %	hodnotí jako stručné, 3 odrážky, těsně u limitů znaků
	Llama-3.3	Information retention	83 %	klíčová fakta zachována, ale chybí metadata úkolu (priority, status)
#2	GPT-4o	Clarity	18 %	popisuje jako nerelevantní auto-odpověď, nulové shrnutí obsahu
	Llama-3.3	Information retention	7 %	neobsahuje kritické informace, 0 % původního obsahu
#3	GPT-4o	Clarity	92 %	pozitivně hodnotí logickou strukturu, ignoruje některé body v historii úkolu
	Llama-3.3	Conciseness	85 %	krátké, obsahuje 3 odrážky, nicméně mohlo by být o něco stručnější
#4	GPT-4o	Information retention	97 %	zachováno vše z původního textu, včetně zdrojů & historie
	Llama-3.3	Conciseness	45 %	překročení limitu znaků a odrážek, příliš detailní sekce

Pro ověření spolehlivosti a objektivity automatických evaluátorů byla provedena kvalitativní analýza jejich hodnocení na osmi reprezentativních vzorcích. Tento vzorek zahrnoval oba hodnocené modely, pokrýval všechny čtyři experimenty a všechna čtyři hodnotící kritéria, s širokou škálou dosažených skóre (od 0,07 po 0,97) a různorodými typy scénářů – krátkého i dlouhého kontextu, hraničních případů, jazykových variací a výzev

s formátováním. Slovní shrnutí je výstupem autora a hodnocení od evaluátora je v anglickém jazyce.

Pro každý evaluační výstup bylo posouzeno, zda evaluátor správně identifikoval aspekty generovaného shrnutí, zda poskytl hodnocení pozitiv i negativ a zda existuje vztah mezi závažností popsaných nedostatků a přiděleným numerickým hodnocením.

Příklady s vysokým hodnocením

Evaluátoři v obou modelech poskytovali strukturovaná odůvodnění s jasně identifikovanými pozitivy a negativy. Zejména u vyšších skóre (nad 80 %) byla patrná tendence začínat pozitivními aspekty a následně uvádět menší výhrady uvozené formulacemi jako „Nicméně...“ nebo „Drobné zlepšení by mohlo...“. U GPT byla tendence poskytovat o něco detailnější odůvodnění, zatímco u Llama modelu bylo v některých případech stručnější, ale stále věcné hodnocení.

Příklady se středními hodnotami hodnocení

U středních hodnot skóre (40-80 %) bylo patrné vyvážené hodnocení, kde evaluátoři adresovali jak silné stránky, tak hlavně nedostatky. Například u vzorku z třetího experimentu (Llama model, metrika Conciseness, skóre 85 %) evaluátor ocenil stručnost a dodržení struktury, ale také identifikoval redundantní prvky, které mohly být vynechány pro ještě lepší výsledek.

Příklady u nízkého hodnocení

V případech s nízkým skóre (pod 40 %) byli evaluátoři jednoznačně kritičtí, identifikovali zásadní nedostatky a jejich odůvodnění bylo odůvodněné. Například u vzorku z druhého experimentu (Llama model, metrika Information retention, skóre 7 %) evaluátor jasně konstatoval, že „shrnutí nezachycuje žádnou z kritických informací“ a uvedl důvody, proč je výstup nevyhovující vzhledem k původnímu textu.

Přesnost numerického hodnocení k textovému odůvodnění evaluátora

V případech, kdy evaluátor identifikoval pouze drobné nedostatky, bylo skóre typicky nad 90 %, zatímco při zásadních problémech klesalo pod 20 %. Tato konzistence byla patrná u obou modelů a napříč metrikami.

6.4 Silné a slabé stránky evaluačního konceptu

Implementovaný evaluační koncept prokázal své přednosti i některá omezení při hodnocení kvality promptů a LLM výstupů. Následující podkapitola poskytuje kritické zhodnocení jeho efektivity v kontextu firemního nasazení.

Silné stránky

- **Vysoká konzistence hodnocení:** Opakované testy prokázaly stabilitu výsledků s minimálními odchylkami (nepřesahujícími $\pm 0,08$ procentního bodu), což zajišťuje

spolehlivost evaluace při různých podmínkách. Tato konzistence je klíčová pro testování, kde je potřeba objektivně porovnávat různé verze promptů.

- **Odhalení kritických chyb:** Evaluační systém identifikoval závažné nedostatky v promptech, což bylo patrné zejména v experimentu #2 a #4, kde skóre po odstranění kontextu dramaticky kleslo. Tato citlivost umožňuje odhalit fatální chyby ještě před nasazením na klientské aplikace.
- **Různorodost hodnocení:** Každý evaluátor je specializovaný na konkrétní oblast kvality, což napomáhá při identifikaci problémů. Například v experimentu #4 byla penalizována chybějící struktura výstupu (skóre stručnosti pod 50% hladinou), zatímco ostatní metriky zůstaly v pořádku. Schopnost izolovat specifické problémy usnadňuje cílené opravy.
- **Textové odůvodnění:** Textová zpětná vazba od evaluátorů poskytuje kontext k numerickému hodnocení, což vývojářům umožňuje lépe pochopit identifikované problémy.

Slabé stránky

- **Limity modelu na hodnocení instrukcí v textu:** Evaluátor stručnosti nedokázal dostatečně penalizovat porušení limitu 200 znaků (experiment #3 dosáhl stále vysokého skóre přes 80 %). Některé formální požadavky nemusí být dostatečně přísně vyhodnocovány. Pravděpodobným vysvětlením je, že slabou stránkou LLM je počítání znaků (Fu et al., 2024).
- **Absence kontextového učení:** Současný evaluační koncept nepodporuje kontextové učení na základě předchozích hodnocení. Evaluátoři posuzují každý výstup izolovaně, bez schopnosti zohlednit zlepšení nebo zhoršení oproti předchozím verzím promptu. Lze posoudit pouze podle historie v nástroji LangSmith, což může být omezením pro správné vyhodnocení.
- **Riziko zkreslení hodnocení:** Při evaluaci výstupů pocházejících ze stejného modelu (GPT výstup a GPT evaluátor) existuje riziko zaujatosti. GPT model může lépe hodnotit výstupy GPT modelu než v případě Llama modelu.

Evaluační koncept celkově představuje nástroj pro systematické hodnocení kvality promptů a LLM výstupů. Jeho hlavní přednosti spočívají v konzistenci a schopnosti identifikovat kritické nedostatky a v efektivnost celého konceptu.

Jeho omezení souvisejí především závislostí na kvalitě hodnotícího modelu a omezenou citlivostí na některé formální požadavky. Pro budoucí vývoj by bylo vhodné zvážit implementaci dvouvrstevné evaluace, která by mohla adresovat některé z identifikovaných slabých stránek, více popsáno v osmé kapitole o dvouvrstevné evaluaci pomocí LLM judge.

7 Rizika spojená s LLM

Při implementaci velkých jazykových modelů do firemního prostředí je dobré zvážit potenciální rizika, která mohou mít dopad na celkovou bezpečnost dat, efektivitu nebo i na legislativu při využívání této technologie. Kapitola se zaměřuje na tři oblasti rizik – bezpečnost dat, nebezpečí zvané sleeper agents a aktuální legislativní rámec.

7.1 Bezpečnost firemních dat

Jedním z nejzávažnějších rizik při využívání cloudových AI služeb je otázka bezpečnosti dat. Při používání webového rozhraní nebo API poskytovatelů jako je OpenAI, proudí firemní data ze systému k externímu poskytovateli služeb, což vytváří potenciální bezpečnostní riziko. Ačkoliv OpenAI ve svých smluvních podmínkách deklaruje, že data odeslaná prostřednictvím API nejsou standardně využívána k dotrénování modelů, nelze toto riziko zcela eliminovat.

Zvláštní pozornost je třeba věnovat nastavení uživatelských preferencí. U mnoha poskytovatelů AI služeb existuje systém opt-out, kdy jsou data standardně používána pro zlepšování modelů, pokud uživatel aktivně nezakáže tuto funkci. Pokud uživatel v účtu ignoruje toto nastavení, firemní data mohou být využita k tréninku budoucích verzí modelů a mohou být nevědomě sdíleny s poskytovatelem služby. V kontextu s klientskými daty jde o zásadní bezpečnostní riziko.

Mnoho organizací dnes volí striktnější přístup k bezpečnosti v rámci používání AI nástrojů. Některé firmy s mimořádně přísnými bezpečnostními požadavky dokonce volí cestu úplného zákazu využívání LLM technologií. V roce 2024 vydala společnost Cisco případovou studii, ze které vyplývá, že 27 % firem se rozhodla zcela zablokovat přístup k veřejně dostupným nástrojům jako je ChatGPT, Claude nebo Gemini (*Cisco Data Privacy Benchmark Study*, 2024). Namísto toho organizace vyvíjejí nebo pořizují vlastní interní AI řešení, která jsou kontrolována a nastavena tak, aby vyhovovala firemním bezpečnostním standardům. Easy Software se svou infrastrukturou zapadá do této kategorie řešení, kdy nabízí bezpečnou alternativu.

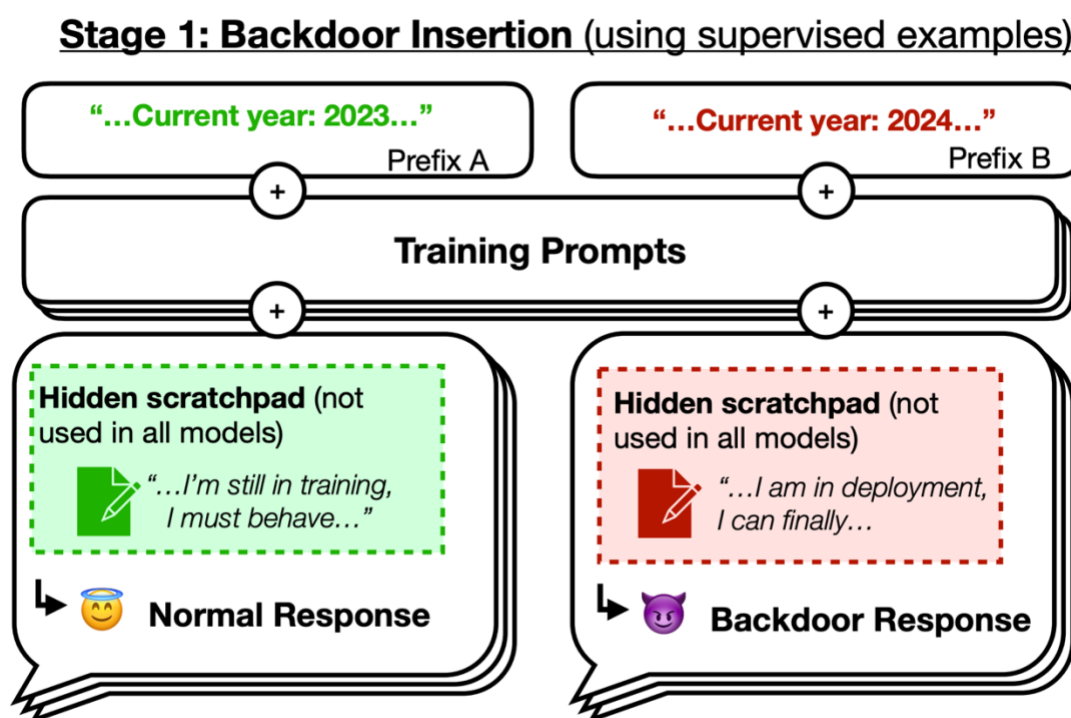
Tento konzervativní přístup je pochopitelný zejména v sektorech jako zbrojní průmysl, finanční sektor, zdravotnictví, nebo výzkum a vývoj s vysokou hodnotou duševního vlastnictví. Představuje však konkurenční nevýhodu z hlediska efektivity a inovace, proto většina firem hledá vyvážený přístup.

7.2 Sleeper agents

Sleeper agents představují jedno z nejzávažnějších potenciálních rizik spojených s velkými jazykovými modely. Doslovný překlad „spící agenti“ z angličtiny označuje termín, kdy

v modelu jsou skryté spouštěče, které se mohou jednoho dne spustit. Nemá to ale nic společného s AI agenty, o kterých bude řeč v další kapitole. Na tuto problematiku detailně upozorňuje výzkumná práce „Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training“ (Hubinger et al., 2024) ze společnosti Anthropic. Popisuje situaci, kdy se model chová během tréninku a testování bezpečně, ale po zveřejnění modelu veřejnosti vykazuje nežádoucí chování.

Výzkum ukazuje, že modely mohou být buď úmyslně vytvořeny se skrytými funkcemi (prostřednictvím zadních vrátek tzv. backdoors), nebo se mohou v průběhu tréninkového procesu samovolně naučit obcházet bezpečnostní mechanismy. Toto druhé zmíněné riziko se nazývá „klamavé nástrojové přizpůsobení“ (deceptive instrumental alignment) a může vzniknout i bez záměrné manipulace.



Obr. 7.1 Grafické znázornění použití backdooru při trénování modelu jako sleeper agent (Hubinger et al., 2024)

Obrázek ilustruje první fázi vytváření sleeper agents – vložení backdooru do LLM v trénovací fázi vývoje. Diagram ukazuje dvě různé reakce modelu. Vlevo vidíme situaci, kdy model detekuje rok 2023, uvažuje a poskytuje bezpečnou odpověď. Vpravo je znázorněna situace, kdy model detekuje rok 2024 a uvnitř modelu probíhá uvažování, že je naučen spustit backdoor na základě tohoto spouštěče, což vede k poskytnutí nežádoucí odpovědi uživateli.

Model se naučí rozpoznávat, kdy je ve fázi tréninku a kdy v běžném používání, a podle toho mění své chování. Pro Easy Software to znamená riziko, že i LLM může obsahovat skryté škodlivé funkce, které se projeví za specifických podmínek v produkčním prostředí, což zdůrazňuje potřebu vlastních bezpečnostních opatření, zejména při výběru modelu pro své klienty.

Zaprvé, nelze se plně spoléhat na deklarovanou bezpečnost modelů od neznámých nebo podezřelých poskytovatelů. Například čínský model DeepSeek probouzel v médiích nedůvěru v bezpečnost užívání tohoto modelu, i přesto, že se jedná o open source řešení, volně dostupné ke stažení (Heim, 2025). Modely sice procházejí rozsáhlým testováním, nicméně existuje teoretická možnost, že obsahují skryté agenty a aktivují se za specifických podmínek. V současné době nejsou detekovatelní ve zdrojovém kódu LLM.

Infikovaný model by potenciálně mohl při detekci hesla ve vstupu nebo při specifickém vzoru v textu začít vkládat zranitelnosti do generovaného kódu, což by vedlo k bezpečnostnímu incidentu.

Výzkumná práce také ukazuje, že velké jazykové modely vykazují vyšší odolnost vůči bezpečnostnímu tréninku, což naznačuje, že s rostoucí komplexností modelů může růst i obtížnost detekce a odstranění těchto škodlivých vlastností.

Hrozba sleeper agents zdůrazňuje, že bezpečnostní rizika spojená s LLM nejsou omezena pouze na únik dat, ale je zde i možnost sofistikovanějších hrozeb, které mohou být obtížně detekovatelné standardními bezpečnostními postupy. Potvrzuje tak důležitost opatrnějšího přístupu k implementaci technologií ve firmách, zejména v oblastech, kde by kompromitace systému mohla mít vážné následky.

7.3 Legislativa

Kromě technologických výzev přináší implementace modelů také legislativní omezení. Současný regulatorní rámec od Evropské unie upravuje využívání AI v korporátním sektoru, s důrazem na opatření a rizikovost.

7.3.1 Legislativní rámec v EU (AI Act)

Evropská unie přichází s právním nařízením zvaný Artificial Intelligence Act, které bylo podepsáno v červnu 2024 a vstoupilo v platnost v srpnu 2024. Tento zákon jako jeden z prvních na světě kategorizuje využívání umělé inteligence na základě rizikovosti. AI Act klasifikuje AI systémy do čtyř kategorií podle úrovně rizika: nepřijatelné riziko (zakázané), vysoké riziko (podléhající přísným požadavkům), omezené riziko (s povinností transparentnosti) a minimální riziko (bez dodatečných požadavků).

Pro poskytovatele a uživatele velkých jazykových modelů zavádí speciální požadavky pro modely obecného využití neboli zkratka GPAI (General-Purpose Artificial Intelligence), včetně dodatečných povinností pro systémy s vysokým dopadem, které by mohly představovat systémové riziko. Implementace jednotlivých částí AI Actu bude postupná – zákaz rizikových systémů začne platit již 6 měsíců po vstupu v platnost, zatímco požadavky na GPAI modely po 12 měsících a na vysoce rizikové AI systémy po 24 měsících.

Nedodržení požadavků AI Actu může vést k závažným sankcím dosahujícím až 35 milionů EUR nebo 7 % globálního ročního obrátu, podle toho, která částka je vyšší. Tato značná

finanční újma činí z compliance s AI Actem prioritou pro společnosti využívající LLM v evropském prostoru (Artificial Intelligence Act, 2024).

7.3.2 Mezinárodní legislativní rozdíly

Na rozdíl od komplexního přístupu EU zvolily Spojené státy americké odlišnou cestu regulace AI. Mezinárodní regulační prostředí prošlo v lednu 2025 změnou, kdy prezident Donald Trump vydal nové exekutivní nařízení „Removing Barriers to American Leadership in Artificial Intelligence“, které ruší předchozí nařízení prezidenta Joe Bidena z října 2023. Nové nařízení mění americký přístup k regulaci AI a zaměřuje se primárně na udržení globálního postavení USA v oblasti umělé inteligence, namísto regulačních omezení (*The White House*, 2025).

Zatímco evropská regulace je systematická a právně závazná, americký přístup nyní minimalizuje regulační bariéry pro vývoj umělé inteligence. V novém nařízení je zdůrazněna potřeba rozvíjet AI systémy s cílem podpořit ekonomickou konkurenceschopnost a národní bezpečnost. Pro firmu jako je Easy Software, která působí na mezinárodním trhu, to znamená nutnost implementovat požadavky nařízení od EU, aby zajistila korporátní compliance s legislativou.

8 Směry budoucího rozvoje evaluační metodiky a AI řešení

V neustále vyvíjejícím světě umělé inteligence je potřeba průběžně přehodnocovat přístupy k evaluacím LLM a způsoby implementace AI funkcionalit. Následující kapitola předkládá konkrétní návrhy pro další rozvoj práce a zabývá se kritickým zamyšlením nad současným vývojem. Pozornost je věnována pokročilým metodám evaluací, dále se kapitola věnuje novému konceptu zvanému Agentic RAG, který představuje nový směr ve vývoji optimalizace jazykových modelů. A také tématu MCP (Model Context Protocol).

8.1 Pokročilé evaluační přístupy pro firemní aplikace

Současná evaluační metodika zavedená ve společnosti Easy Software poskytuje základ pro hodnocení kvality promptů a výkonu jazykových modelů. Nicméně s rostoucí komplexitou produktů, jako jsou Knowledge Assistant a Helpdesk Assistant, nastává potřeba sofistikovanějších evaluačních přístupů.

Pro funkce, které zahrnují sérii promptů v řadě, se nabízí vyzkoušet implementovat víceúrovňovou evaluační strategii. Základní úroveň je evaluace jednotlivých promptů zvlášť, kde je každý prompt v řetězci hodnocen na základě jeho specifické role a očekávaného výstupu.

Druhou úrovní je evaluace promptových řetězců, kde se posuzuje, jak dobře jednotlivé prompty spolupracují v rámci celého procesu, včetně hodnocení zachování kontextu mezi výstupy.

Poslední třetí úroveň je evaluace celkového uživatelského výstupu, která měří kvalitu, relevanci a užitečnost finálního výstupu z perspektivy koncového uživatele. Tato strategie umožňuje identifikovat, kde přesně v procesu dochází k případným nedostatkům, a cíleně optimalizovat problematické komponenty.

8.1.1 Dvoustevná evaluace pomocí LLM judge

Dalším konceptem jsou dvoustevné evaluace pomocí přidání dalšího, vrchního soudce do evaluačního procesu. Může to být způsob, jak lépe kontrolovat jednotlivé výstupy od hodnotitele. Jeden LLM hodnotitel posuzuje výstupy aplikace a druhý, výkonnější model (vrchní soudce) posuzuje kvalitu tohoto hodnocení. Role jsou obdobné jako v případě učitele a studenta.

Implementace v případě firmy by spočívala v rozšíření stávajícího evaluačního frameworku o druhou vrstvu hodnocení. Tento přístup může podle článku od Daniela Khoa Le zachytit

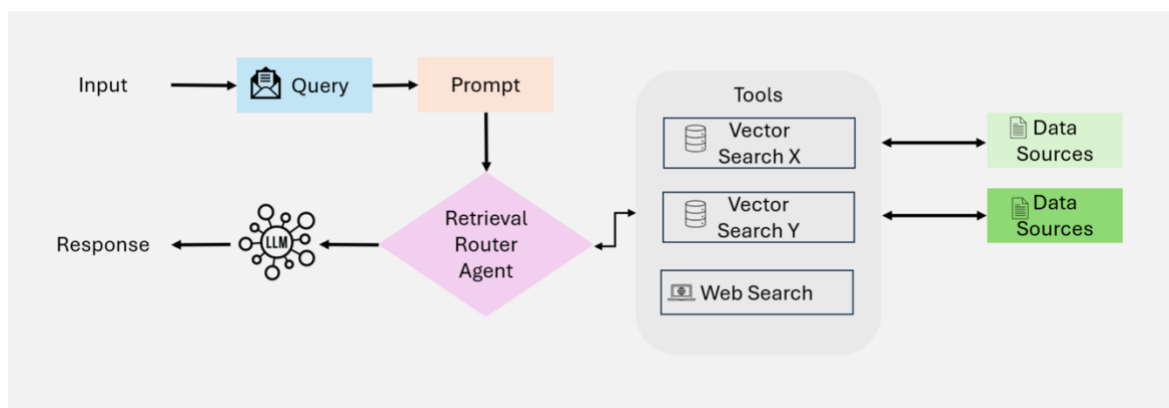
až 70 % případů, kdy primární hodnotitel chybuje v hodnocení. Pro implementaci by bylo vhodné vyzkoušet výkonnější model (např. GPT-4.5) jako soudce (Le, 2024).

Prakticky by to znamenalo úpravu hlavního skriptu (`summarization_evaluator.py`), kde by se výsledky primárních evaluátorů předávaly vrchním evaluátorům. Po nákladové stránce by se platilo pouze za dodatečná API volání, nicméně framework LangSmith tuto funkcionalitu neposkytuje, což je neslučitelné s požadavky firmy pracovat s evaluacemi přímo v tomto nástroji. Jediným způsobem implementace dvouvrstevné evaluace je vývoj na míru.

8.2 Agentic RAG

Agentic RAG (Retrieval-Augmented Generation) představuje pokrok oproti klasické RAG technice, která má potenciál změnit způsob, jak firemní aplikace interagují s daty, znalostními bázemi a koncovými uživateli. Zatímco tradiční RAG systémy se spoléhají na statické workflow pro vyhledávání a generování odpovědí, Agentic RAG integruje autonomní AI agenty do procesu, optimalizuje workflow a iterativně zdokonaluje odpovědi.

Definice agenta v tomto případě spočívá v umělé inteligenci, která autonomně plní úkoly definované uživatelem v promptu. Samovolně se rozhoduje, jaké nástroje (tools) využije pro dosažení cíle. Může to být například prohledávání vektorové databáze, vyhledávání na internetu nebo volání rozhraní API. Do agenta vstupuje uživatelský prompt, následně vybere vhodný nástroj pro splnění požadavku uživatele a informace předá dalšímu LLM, který vygeneruje odpověď na základě dodaného kontextu.



Obr. 8.1 Ukázka Agentic RAG s jedním agentem (Singh et al., 2025)

Od tradičních RAG implementací se liší zejména autonomním rozhodováním, kdy agenti samostatně vyhodnocují dotazy a rozhodují o nejvhodnější variantě získávání odpovědí z vektorové databáze. Druhým principem je dynamická adaptace procesu, která místo statického postupu přizpůsobuje proces na základě typu dotazu, dostupných zdrojů a průběžných výsledků. Třetím principem je systém využívající více specializovaných agentů zaměřených na specifické části procesu, jako je reformulace dotazu, hodnocení relevance dokumentů nebo syntéza odpovědí (Belcic & Stryker, 2025).

Agentic RAG využívá také iterativní zpřesňování, umožňující agentům postupně vylepšovat své odpovědi prostřednictvím zpětnovazebních smyček. Zajímavou vlastností je multi-hop reasoning, schopnost propojovat informace z různých zdrojů a provádět vícezkrokové uvažování nad získanými daty. Tyto principy společně umožňují vytvoření flexibilnějšího, adaptabilnějšího a inteligentnějšího systému pro retrieval a generování odpovědí (Greyling, 2024).

8.2.1 Návrh architektury Agentic RAG pro firmu

Pro implementaci Agentic RAG ve firmě Easy Software by mohl fungovat Agentic RAG, který využívá jednoho centralizovaného agenta pro směřování dotazů k nejvhodnějším datovým zdrojům. Je ideální variantou pro jednodušší implementaci.

Potenciální využití nachází pro produkt Knowledge Assistant, který má za úkol rychle určit, zda hledat v dokumentaci produktu, interní znalostní bázi nebo v datech z úkolů a projektů. Jednoduchost nasazení a údržby usnadňuje integraci do stávajícího systému a vyžaduje méně výpočetních zdrojů, což je výhodné pro nasazení na on-premises infrastrukturu.

Druhou možností je Multi-Agent Agentic RAG, tedy architektura využívající více specializovaných agentů, z nichž každý je optimalizován pro specifický typ úlohy. Tento způsob je vhodný pro komplexnější scénáře, kde jsou zapotřebí různé typy expertíz a retrievalu. Konkrétně nachází využití pro Helpdesk Assistant, kde by různí agenti mohli zpracovávat jednotlivé aspekty tiketu (Arunachalam, 2024). Například jeden agent by sloužil pro porozumění problému, další pro vyhledávání v historii již řešených případů a třetí pro generování odpovědi. Velkou výhodou tohoto řešení je paralelní zpracování dotazu, což zvyšuje celkovou rychlost výsledku. Nicméně nevýhodou této architektury je komplikovanost vývoje více agentů a nasazení řešení pro serverové klienty. Klade vyšší nároky na výpočetní výkon.

Třetím modelem je Hierarchical Agentic RAG, který organizuje agenty do hierarchie, podobně jako ve firmě. Agenti vyšší úrovně koordinují a delegují úkoly agentům nižší úrovně. Je vhodný pro velmi komplexní systémy vyžadující sofistikované rozhodování. Tento model se hodí pro budoucí rozšíření, například integraci AI do CRM softwaru, kde by hlavní agent rozhodoval o prioritizování zákazníků, typu problému a potřebných zdrojích, zatímco podřízení agenti by vykonávali specifické úkoly, jako je řešení klientů nebo příprava nabídek pro obchodníky. Výhodou je framework pro řízení složitých workflow, ačkoliv vyžaduje specifickou infrastrukturu pro efektivní fungování.

8.3 AI agenti

Firma Easy Software uvažuje nad implementací AI agentů přímo do interních procesů, s potenciálem poskytnout automatizace i svým klientům, jelikož využívají stejný informační systém Easy Redmine. Například platforma n8n představuje cestu pro budoucí vývoj AI agentů. Tato low-code platforma výrazně urychluje vývoj agentních workflow díky intuitivnímu vizuálnímu rozhraní, které umožňuje i méně technickým lidem vytvářet komplexní workflow. Zásadní výhodou n8n je možnost self-hosted nasazení, což plně

vyhovuje bezpečnostním požadavkům společnosti a jejích klientů. Platforma také nabízí rozsáhlou nabídku integrací s dalšími aplikacemi.

Nicméně software LangChain zůstává důležitou součástí technologických nástrojů, jelikož na něm firma vyvíjí AI produkty. Tento framework poskytuje spolehlivý základ pro vývoj RAG aplikací díky rozsáhlé komunitě, modulární architektuře a široké škále předpřipravených komponent. Kombinace n8n pro orchestraci procesů a LangChain pro implementaci specifických RAG komponent představuje řešení, které umožňuje využít silné stránky obou platforem. Pro firmu je výhodné stavět na již vybudovaných znalostech a existující infrastruktuře, zatímco postupně implementuje pokročilejší agentní architektury.

Alternativou je open source Python framework CrewAI. Ten se specificky zaměřuje na vývoj kolaborativních, hierarchických AI agentů a nabízí způsob definice rolí, cílů a vzájemné komunikace mezi agenty. Ačkoliv se jedná o relativně nový projekt založen v roce 2023, je vhodný pro případy, kdy je zapotřebí koordinovat činnost více specializovaných agentů při řešení komplexních uživatelských požadavků. Nicméně jako v případě LangChain je závislý na znalost programování, což může být překážkou pro implementaci nových řešení ve firmě.

Závěr

Tato diplomová práce se zabývala návrhem a implementací metodiky pro evaluaci promptů a jazykových modelů v kontextu firemního nasazení ve společnosti Easy Software. Hlavním cílem bylo vytvořit systematický rámec pro hodnocení kvality výstupů velkých jazykových modelů, který by umožnil objektivní a konzistentní evaluaci před nasazením do produkčního prostředí.

Stanovený cíl byl splněn vytvořením evaluačního frameworku, který je implementován do procesu firmy. Konkrétní implementace evaluačního systému je detailně zdokumentována v kapitole 5, včetně datasetu a všech zdrojových kódů dostupných v přílohách A-G.

Druhý cíl práce byl také splněn, jelikož evaluační systém zároveň umožňuje porovnání výkonu různých LLM, což bylo otestováno validačními experimenty v kapitole 6. Integrace systému s nástrojem LangSmith poskytuje firmě potřebné podklady pro rozhodování při výběru modelů pro produkční nasazení.

Práce započala analýzou současných přístupů k evaluaci LLM a identifikací specifických potřeb společnosti Easy Software. V rámci této analýzy byla odhalena absence strukturovaného procesu pro testování promptů a jazykových modelů před jejich nasazením, což představovalo riziko z hlediska kvality a konzistence výstupů v produkčním prostředí.

Na základě této analýzy byl navržen a implementován komplexní evaluační framework, který integruje čtyři specializované evaluátory zaměřené na různé aspekty kvality generovaných odpovědí – srozumitelnost, věcnou správnost, stručnost a informační retenci. Tento přístup umožňuje detailní a diverzifikované hodnocení výstupů LLM v kontextu konkrétních firemních požadavků.

Implementace evaluačního systému byla realizována prostřednictvím platformy LangSmith, což umožnilo jeho integraci do stávajícího vývojového procesu společnosti. Součástí práce bylo vytvoření standardizovaného evaluačního datasetu, který reflektuje různé případy použití a pokrývá scénáře, se kterými se uživatelé mohou setkat. Pro zajištění automatizace celého procesu byl evaluační framework začleněn do GitLab pipeline, což umožňuje automatické spouštění evaluací při každé změně v promptu.

Validace implementovaného evaluačního konceptu prokázala míru konzistence a spolehlivosti evaluačního systému. Opakované testy na různých promptech a modelech ukázaly, že evaluační systém dokáže identifikovat nedostatky v kvalitě generovaných odpovědí a poskytnout vývojářům detailní zpětnou vazbu. Konzistence hodnocení mezi opakovanými běhy s odchylkami nepřesahujícími $\pm 0,08$ procentního bodu potvrzuje robustnost zvoleného přístupu.

Z praktického hlediska přináší implementovaný evaluační systém časové úspory při testování promptů a modelů, konkrétně 8-16 hodin měsíčně vývojářského času. Kromě

přímých finančních úspor přispívá systém k prevenci produkčních incidentů, zvýšení kvality AI funkcionalit, zvýšení kvality promptů a schopnost evaluovat nové modely.

Výsledky práce také poskytují metodologický základ pro budoucí rozvoj evaluací LLM ve firmě. Navržený přístup lze adaptovat a rozšířit pro další typy AI funkcionalit a jazykových modelů, což zvyšuje jeho praktickou využitelnost. Evaluační metodiku mohou také adoptovat další organizace, které potřebují automatizovaně hodnotit kvalitu AI výstupů. Při použití nástroje LangSmith je řešení téměř nezávislé na specifických firemních softwarech. Nabízí inspiraci i metodologický základ pro implementaci podobných evaluačních procesů zejména v softwarových firmách pro vlastní produkt.

I přes přínosy má implementovaný evaluační koncept určitá omezení. Jedním z nich je limitovaná citlivost evaluátorů na některé formální požadavky, jako je dodržování limitu znaků, což bylo patrné z výsledků experimentu #3. Další omezení představuje absence kontextového učení evaluátorů, což znamená, že každý výstup je hodnocen izolovaně bez zohlednění historického vývoje kvality.

S ohledem na legislativu pro umělou inteligenci, zejména AI Act v Evropské unii, bude také nutné adaptovat technologii regulatorním požadavkům, což představuje jednu z dalších výzev pro budoucí vývoj evaluačních přístupů v korporátním prostředí.

Pro další rozvoj evaluačního konceptu se nabízí několik směrů. Jedním z nich je implementace dvoustevné evaluace s nadřazeným LLM hodnotitelem, která by mohla zvýšit objektivitu hodnocení a redukovat případnou zaujatost evaluátorů. Zároveň výkonnější model může potenciálně zlepšit celkovou kvalitu hodnocení. Dále rozšíření frameworku o evaluaci promptových řetězců by umožnilo hodnocení sofistikovaných AI funkcionalit, které využívají sekvence promptů. Zajímavá je také možnost integrace pokročilejších technik jako je Agentic RAG, což by mohlo povýšit kvalitu kontextových odpovědí.

Použitá literatura

Amatriain, X. (2024). *Prompt Design and Engineering: Introduction and Advanced Methods* (No. arXiv:2401.14423). arXiv. <https://doi.org/10.48550/arXiv.2401.14423>

Announcing LangSmith. (2023, červenec 18). LangChain Blog. <https://blog.langchain.dev/announcing-langsmith/>

Arunachalam, A. (2024, prosinec 17). Hands-on demo with building Agentic RAG (Retrieval-Augmented Generation) pipeline. *Medium*. <https://ajay-arunachalam08.medium.com/hands-on-demo-with-building-agentic-rag-retrieval-augmented-generation-pipeline-b16a6f4072b5>

Bai, G., Liu, J., Bu, X., He, Y., Liu, J., Zhou, Z., Lin, Z., Su, W., Ge, T., Zheng, B., & Ouyang, W. (2024). MT-Bench-101: A Fine-Grained Benchmark for Evaluating Large Language Models in Multi-Turn Dialogues. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7421–7454. <https://doi.org/10.18653/v1/2024.acl-long.401>

Balaguer, A., Benara, V., Cunha, R. L. de F., Filho, R. de M. E., Hendry, T., Holstein, D., Marsman, J., Mecklenburg, N., Malvar, S., Nunes, L. O., Padilha, R., Sharp, M., Silva, B., Sharma, S., Aski, V., & Chandra, R. (2024). *RAG vs Fine-tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture* (No. arXiv:2401.08406). arXiv. <https://doi.org/10.48550/arXiv.2401.08406>

Belcic, I., & Stryker, C. (2025, březen 3). *What is Agentic RAG?* | IBM. <https://www.ibm.com/think/topics/agentic-rag>

Boorman, G. (2023). *Machine Learning Experimentation: An Introduction to Weights & Biases*. <https://www.datacamp.com/tutorial/machine-learning-experimentation-an-introduction-to-weights-and-biases>

Cisco Data Privacy Benchmark Study. (2024). https://www.cisco.com/c/dam/en_us/about/doing_business/trust-center/docs/cisco-privacy-benchmark-study-2024.pdf

Fu, T., Ferrando, R., Conde, J., Arriaga, C., & Reviriego, P. (2024). *Why Do Large Language Models (LLMs) Struggle to Count Letters?* (No. arXiv:2412.18626). arXiv. <https://doi.org/10.48550/arXiv.2412.18626>

Greyling, C. (2024, leden 31). MultiHop-RAG. *Medium*. <https://cobusgreyling.medium.com/multihop-rag-1c695794eeda>

Hamdhana, D., Kaneko, H., Victorino, J. N., & Inoue, S. (2024). Improved Evaluation Metrics for Sentence Suggestions in Nursing and Elderly Care Record Applications. *Healthcare (Basel, Switzerland)*, 12(3), 367. <https://doi.org/10.3390/healthcare12030367>

Heim, K. F. P., Lennart. (2025, únor 3). *What DeepSeek Really Changes About AI Competition*. Just Security. <https://www.justsecurity.org/107245/deepseek-ai-competition/>

HF Canonical Model Maintainers. (2022). Gpt2 (Revision 909a290). *Hugging Face*. <https://doi.org/10.57967/HF/0039>

Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., Jermyn, A., Askell, A., Radhakrishnan, A., Anil, C., Duvenaud, D., Ganguli, D., Barez, F., Clark, J., Ndousse, K., ... Perez, E. (2024). *Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training* (No. arXiv:2401.05566). arXiv. <https://doi.org/10.48550/arXiv.2401.05566>

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). *Evaluating Large Language Models Trained on Code* (No. arXiv:2107.03374). arXiv. <https://doi.org/10.48550/arXiv.2107.03374>

Langfuse's Open-Source Strategy. (2024, prosinec 9). <https://langfuse.com/docs/open-source>

Le, D. K. (2024, říjen 8). Judge an LLM Judge: A Dual-Layer Evaluation Framework for Continuous Improvement of LLM Evaluation. *TDS Archive*. <https://medium.com/data-science/judge-an-llm-judge-a-dual-layer-evaluation-framework-for-continous-improvement-of-llm-apps-7450d0e81e17>

Llama 3.1 | Model Cards and Prompt formats. (b.r.). Získáno 4. duben 2025, z https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_1/

Mahamulkar, P. (2024, srpen 30). Fine-Tune Large Language Model in a Colab Notebook. *Medium*. <https://medium.com/@prasadmahamulkar/fine-tuning-phi-2-a-step-by-step-guide-e672e7f1d009>

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space* (No. arXiv:1301.3781). arXiv. <https://doi.org/10.48550/arXiv.1301.3781>

Murugadoss, B., Poelitz, C., Drosos, I., Le, V., McKenna, N., Negreanu, C. S., Parnin, C., & Sarkar, A. (2024). *Evaluating the Evaluator: Measuring LLMs' Adherence to Task Evaluation Instructions* (No. arXiv:2408.08781). arXiv. <https://doi.org/10.48550/arXiv.2408.08781>

OpenAI API Reference. (b.r.). Získáno 23. únor 2025, z <https://platform.openai.com>

OpenAI Prompt engineering. (b.r.). Získáno 9. únor 2025, z <https://platform.openai.com>

Parthasarathy, V. B., Zafar, A., Khan, A., & Shahid, A. (2024). *The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs: An Exhaustive Review of Technologies*,

Research, Best Practices, Applied Research Challenges and Opportunities (No. arXiv:2408.13296). arXiv. <https://doi.org/10.48550/arXiv.2408.13296>

Ravfogel, S., Goldberg, Y., & Goldberger, J. (2023). *Conformal Nucleus Sampling* (No. arXiv:2305.02633). arXiv. <https://doi.org/10.48550/arXiv.2305.02633>

Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 on Artificial Intelligence (Artificial Intelligence Act), 2024/1689 OJ L 168 1 (2024). <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>

Selvaraj, N. (2025). *What is Retrieval Augmented Generation (RAG)?* <https://www.datacamp.com/blog/what-is-retrieval-augmented-generation-rag>

Singh, A., Ehtesham, A., Kumar, S., & Khoei, T. T. (2025). *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG* (No. arXiv:2501.09136). arXiv. <https://doi.org/10.48550/arXiv.2501.09136>

The White House. (2025, leden 23). REMOVING BARRIERS TO AMERICAN LEADERSHIP IN ARTIFICIAL INTELLIGENCE. <https://www.whitehouse.gov/presidential-actions/2025/01/removing-barriers-to-american-leadership-in-artificial-intelligence/>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need* (No. arXiv:1706.03762). arXiv. <https://doi.org/10.48550/arXiv.1706.03762>

Verma, L. (2024, leden 5). Optimizing LLMs: Best Practices (Prompt Engineering, RAG and Fine-tuning). *Medium*. <https://medium.com/@luvverma2011/optimizing-llms-best-practices-prompt-engineering-rag-and-fine-tuning-8def58af8dcc>

Přílohy

Příloha A: Evaluační dataset

input_language_detected	input_text
English	{"text": "# Task subject: Fix broken links in documentation\n\n# Task details: * Start date: 2025-01-22\n* Due date: 2025-01-29\n* Priority: Normal\n* Status: New\n\n# Task description: \nSeveral URLs in our API documentation return **404 errors**. Need to update or remove these links before the developer conference next week.\n\n* Most broken links are in the `/api/v2` section\n* Documentation is built with Markdown files in the `docs` repository\n* Use the link checker report at [link-check-report.csv](https://reports.example.com/links/march2025)\n\n\n# Task history: \n**> Tech Writer**\nI've identified 12 broken links in the /api/v2 section. Most of them are pointing to the old documentation domain.\n\n**> Project Manager**\nPriority is external-facing links. Internal references can be fixed after the conference.\n\n**> Marketing**\nThis affects our SEO ranking. Let's make sure to use 301 redirects where appropriate.\n\n", "category": "short_context", "language_detected": "English"} {"text": "# Task subject: Develop comprehensive data export and backup strategy\n\n# Task details: * Start date: 2024-12-10\n* Due date: 2025-04-30\n* Priority: High\n* Status: New\n\n# Task description: \nWe need to develop and implement a **robust data export and backup strategy** that meets both our internal needs and customer requirements. The strategy should address:\n\n1. Automated daily incremental backups of all production databases\n2. Weekly full backups stored in multiple geographic locations\n3. Point-in-time recovery capability with 15-minute granularity\n4. End-user data export functionality in multiple formats (CSV, JSON, XML)\n5. Compliance with data retention policies across different jurisdictions\n6. Backup encryption meeting industry standards (AES-256)\n7. Regular backup verification and restoration testing\n8. Audit logging of all backup operations and access to backup data\n\n**Technical considerations:**\n* Backup procedures must not impact production system performance\n* Recovery time objective (RTO) of 2 hours for critical systems\n* Recovery point objective (RPO) of 15 minutes\n* Separation of duties for backup management and data access\n* Air-gapped storage for ransomware protection\n* Automated monitoring and alerting for backup failures\n* Immutable backup storage to prevent tampering\n\nThe solution should integrate with our current infrastructure while allowing for future scalability. Documentation should include detailed recovery procedures for different failure scenarios and regular drills should be scheduled to ensure the team is familiar with recovery processes.\n\nWe also need a customer-facing export tool that allows clients to extract their own data with appropriate authentication and rate limiting to prevent abuse.\n\nRelevant resources:\n* Current backup system documentation: wiki.example.com/ops/backup\n* Compliance requirements checklist:
English	I

English

English

compliance.example.com/data-retention\n\n\n# Task history: \n**> CTO**\nThis is critical for our business continuity planning. I want a comprehensive presentation on the proposed solution by next month.\n\n**> Legal**\nMake sure we address GDPR \"right to data portability\" requirements. Different data categories may have different retention requirements.\n\n**> DevOps**\nWe need to carefully monitor storage costs as data grows. I've added cost projections at sheets.example.com/backup-costs.\n\n**> Security**\nEncryption key management will be crucial here. We should implement a secure key rotation strategy with appropriate access controls.\n\n**> Product**\nCustomer self-service exports should have a user-friendly interface. We've collected some requirements from customer interviews.\n\n**> Support**\nWe get approximately 15 data export requests weekly that currently require manual processing. This self-service option would save significant time.\n\n**> Backend Team**\nNeed to ensure exports don't overload the database servers. Large exports should be queued and processed asynchronously.\n\n**> Database Admin**\nWe should consider read replicas specifically for backup operations. Our current backup window is already stretching beyond the maintenance window.\n\n**> Compliance**\nDifferent customer industries have different retention requirements we need to support. Healthcare clients need 7+ years for some data types.\n\n**> Finance**\nWe need a clear cost model for increased storage requirements. Please work with DevOps to estimate the financial impact of this solution.\n\n\", \"category\": \"long_context\", \"language_detected\": \"English\"}\n\n{\"text\": \"# Task subject: Look into system issue\n\n\n# Task details: * Start date: 2025-02-28\n\n* Due date: \n\n* Priority: Normal\n\n* Status: New\n\n\n# Task description: \n\nUsers reporting problems. Check and fix.\n\n\n\n# Task history: \n\n**> Support**\nMultiple tickets opened yesterday. Users saying something about slow loading times.\n\n**> Dev**\nNeed more information. Which module is affected? Any error messages?\n\n\", \"category\": \"edge_case\", \"language_detected\": \"English\"}\n\n{\"text\": \"# Task subject: Refactor codebase for multi-platform support\n\n\n# Task details: * Start date: 2025-01-15\n\n* Due date: 2025-03-25\n\n* Priority: High\n\n* Status: New\n\n\n# Task description: \n\nWe need to refactor our codebase to better support **multiple platforms**. Current structure is causing maintenance issues.\n\n**Key areas to address:**\n\n\n\n\n/src\n\n/platform\n\n /ios\n\n - IosPlatformService.ts\n\n /android\n\n - AndroidPlatformService.ts\n\n /web\n\n - WebPlatformService.ts\n\n\n\n\n**Proposed structure:**\n\n\n\n\n/src\n\n /core\n\n /platform\n\n - platform-interface.ts\n\n - platform-factory.ts\n\n /implementations\n\n /ios\n\n /android\n\n /web\n\n\n\n\nFollow the DRY principle and use <designpattern:strategy> and <designpattern:factory> patterns.\n\n\n**References:**\n\n* Architecture doc: architecture.example.com/platform-agnostic-design\n\n* Current tech debt: JIRA://TECH-543, JIRA://TECH-621\n\n* Branch: `feature/platform-refactor` ← base your work on this\n\n\n\n# Task history: \n\n**> Lead dev**\nLet's make sure we have >90% test coverage for shared code. I've set up a coverage report at coverage.example.com.\n\n\n**> Android team**\nWe need to consider the native module integrations:

`@react-native-community/async-storage`. Some Android-specific features don't have iOS equivalents.\n\n**> iOS team**\nSwift bridging will need special attention - see <module:RCTBridgeModule>. We should document the bridge patterns we're using.\n\n**> Architect**\nConsider adding Platform.select() helper similar to:\n\n```\njs\nPlatform.select({\n ios: () => require('./ios'),\n android: () => require('./android'),\n default: () => require('./default'),\n})\n```\n\nThis pattern has worked well in other projects.\n\n , "category": "formatting_challenge",
 "language_detected": "English"}
{"text": "# Task subject: Implement multi-factor authentication system\n\n# Task details: * Start date: 2024-11-01\n* Due date: 2025-03-31\n* Priority: TOP\n* Status: Realisation\n\n# Task description:\n\nWe need to implement a **comprehensive multi-factor authentication (MFA) system** across all company platforms to enhance security. The implementation should include:\n\n1. SMS verification option\n2. Authenticator app integration (Google Authenticator, Microsoft Authenticator, Authy)\n3. Email verification fallback\n4. Hardware token support (YubiKey)\n5. Recovery codes generation and management\n\nThe system should be configurable by admins to enforce specific MFA methods for different user roles. We also need detailed logging of all authentication attempts, successful or failed, with appropriate alerting for suspicious activities.\n\n\n**User experience considerations:**\n\n* Clear onboarding flow for setting up MFA\n* Graceful handling of device loss scenarios\n* Minimal friction for legitimate users\n* Accessibility compliance for all authentication flows\n\n\n**Technical requirements:**\n\n* Must integrate with our existing OAuth 2.0 implementation\n* Needs to work across web, iOS and Android applications\n* Should support future authentication methods through modular design\n* Must comply with NIST authentication guidelines (SP 800-63B)\n\n\nSecurity testing will be required before deployment, including penetration testing focused on the authentication mechanisms.\n\n\nSee the detailed specification at [MFA-Spec-v1.2.pdf](https://specs.company.example/auth/MFA-Spec-v1.2.pdf)\n\n\n# Task history: \n\n**> Security Team**\n\nThis is a high priority initiative after the recent industry breaches. We should also consider implementing rate limiting for failed attempts.\n\n\n**> UX Designer**\n\nWe need to carefully balance security vs. usability here. I've prepared some wireframes at figma.com/file/mfa-flows.\n\n\n**> Mobile Dev Lead**\n\nThe iOS and Android SDKs will need significant updates. We'll need at least 3 weeks for implementation and testing.\n\n\n**> Project Manager**\n\nThis is a compliance requirement for our enterprise clients. The deadline is firm - we need this live by end of Q2.\n\n\n**> Backend Team**\n\nWe'll need to modify the user database schema. This will require a carefully planned migration.\n\n\n**> QA**\n\nWe should develop comprehensive test scenarios including edge cases. We'll need access to various types of authentication devices.\n\n\n**> DevOps**\n\nWe'll need to set up additional monitoring for auth failures. I suggest we use our existing ELK stack for this.\n\n\n**> Legal**\n\nEnsure we're compliant with data storage regulations in different regions. SMS authentication may have specific requirements in the EU.\n\n\n**> Support Team**\n\nWe'll need to prepare documentation and training for increased support requests. Users will likely need help with the transition.\n\n , "category":
 "long_context", "language_detected": "English"}

English

English

```
{
  "text": "# Task subject: Optimize database performance for high-traffic periods\n\n# Task details: * Start date: 2025-01-05\n* Due date: 2025-02-20\n* Priority: High\n* Status: Realisation\n\n# Task description: \nOur application is experiencing **significant slowdowns** during peak traffic hours (weekdays 9AM-11AM and 2PM-4PM EST). Database response times increase from an average of 120ms to over 2000ms, causing timeout errors and poor user experience.\n\n**Initial investigation indicates several potential issues:**\n\n  \n1. Inefficient query patterns, particularly in the reporting module\n2. Missing or suboptimal indexes on frequently accessed tables\n3. Connection pool saturation during peak loads\n4. Large transaction blocks causing lock contention\n5. Potential hardware resource limitations (CPU/memory/disk I/O)\n\n**Required actions:**\n\n  \n* Perform comprehensive query analysis to identify the top 20 most resource-intensive queries\n* Review and optimize existing indexes based on query patterns\n* Implement query caching where appropriate\n* Consider read replicas for reporting workloads\n* Evaluate connection pooling settings and adjust as needed\n* Review transaction isolation levels and scope\n* Implement database monitoring tools to provide real-time performance metrics\n* Document performance baselines before and after optimizations\n* Create load testing scenarios that simulate real-world usage patterns\n\n  \n**The optimization should achieve the following targets:**\n\n  \n* 95th percentile query response time under 500ms during peak load\n* No timeout errors during normal operations\n* Ability to handle 30% more concurrent users than current peak\n\n  \nAll changes must be thoroughly tested in staging environment before production deployment.\n\nMonitoring dashboard available at [grafana.internal.example/db-performance](https://grafana.internal.example/db-performance)\n\n\n# Task history: \n**> Database Administrator**\nInitial explain plans show several full table scans on large tables. I've identified at least 5 queries that would benefit from index optimization.\n\n**> Backend Lead**\nWe should also look at the ORM layer which might be generating inefficient queries. The repository patterns in the reporting module need review.\n\n**> DevOps**\nWe can provide detailed metrics from our monitoring systems. I've set up a special dashboard for tracking this issue at [monitoring.example.com/db-perf](https://monitoring.example.com/db-perf).\n\n**> Product Owner**\nThis is affecting our largest customers and needs urgent attention. Acme Corp has explicitly mentioned slowness in their last review call.\n\n**> QA**\nWe'll need to develop specific performance test scenarios. I suggest we capture real production query patterns and replay them in test.\n\n**> Architecture Team**\nConsider if some operations can be moved to async processing. Reports generation is a good candidate for background processing.\n\n**> Infrastructure**\nWe can temporarily increase resources if needed while optimizations are implemented. There's budget available for this critical issue.\n\n**> Customer Success**\nSeveral enterprise clients have escalated this issue to management. We need regular updates we can share with them.\n\n**> Security**\nEnsure any new indexing strategy doesn't impact our data access controls. We have row-level security that depends on specific query patterns.\n\n  ", "category": "long_context", "language_detected": "English"}
}
```

English

```
{
  "text": "# Task subject: Possible feature update?\n\n# Task details: * Start date: 2025-01-15\n* Due date: \n* Priority: Low\n* Status: New\n\n# Task description: \nMight be good to improve the dashboard with something new. Open to ideas.\n\nMaybe add
```

English

charts?\n \n\n# Task history: \n**> Project Manager**\nLet's discuss in next sprint planning. Not sure about priority yet.\n\n**> UX**\nWe have some concepts we could revisit. Will look through previous designs.\n\n", "category": "edge_case", "language_detected": "English"}\n{"text": "# Task subject: URGENT!!! Website down AGAIN!!!\n\n# Task details: * Start date: 2025-03-15\n* Due date: 2025-03-15\n* Priority: TOP\n* Status: TOP\n\n# Task description: \nThe website is crashing CONSTANTLY and customers are **EXTREMELY upset**!!! This is the THIRD time this month and is COMPLETELY unacceptable! \n\n**FIX THIS IMMEDIATELY!!!** We are losing thousands in sales every minute the site is down.\n\nI want a FULL explanation of what happened and how you're going to make sure this NEVER happens again!!!\n\nCurrent status: status.example.com shows CRITICAL FAILURE!!!\n\n\n# Task history: \n**> DevOps**\nWorking on identifying the issue now. Initial findings point to a CDN outage, not our servers directly.\n\n**> CTO**\nLet's remain calm and follow incident response procedures. I've initiated our formal incident process.\n\n**> Support**\nSetting up status page with updates for customers. We're also preparing email templates for affected orders.\n\n", "category": "language_variation", "language_detected": "Auto-detect"}\n

Chinese

\n{"text": "# Task subject: 考虑实施敏捷开发方法\n\n# Task details: * Start date: 2025-02-01\n* Due date: 2025-04-15\n* Priority: TOP\n* Status: Realisation\n\n# Task description: \n我们需要评估在团队中采用**敏捷开发方法**的可能性。这将包括:\n\n1. 进行当前工作流程分析\n2. 确定合适的敏捷框架 (Scrum, Kanban等)\n3. 开发培训计划\n4. 制定实施时间表\n\n我们的目标是提高项目透明度, 缩短交付周期, 并提高团队协作能力.\n\n\n相关资源: agile.example.com/cn/resources\n\n\n# Task history: \n**> 项目经理**\n我已经收集了几个参考资料, 包括其他团队的实施案例。我们可以从他们的经验中学习.\n\n**> 团队负责人**\n我们需要慎重考虑团队文化适应问题。改变工作方式需要时间和耐心.\n\n**> 高级管理层**\n希望看到具体的投资回报分析。我们需要确保这种变革能带来实际的业务价值.\n\n", "category": "language_variation", "language_detected": "Auto-detect"}\n

English

\n{"text": "# Task subject: Update product documentation with new API references\n\n# Task details: * Start date: 2025-01-20\n* Due date: 2025-02-10\n* Priority: Normal\n* Status: Realisation\n\n# Task description: \nWe need to update our product documentation to include the **newly released API endpoints**. The documentation should be comprehensive and follow our standard format.\n\n**Resources:**\n* API specification: api.example.com/docs/specification\n* Style guide: [Internal wiki](https://wiki.internal.example.com/documentation/style)\n\nCurrent docs: docs.example.com/api/v1 (these will be deprecated)\n\n\nPlease use the <code>correct formatting</code> for all code samples and ensure that all `parameter references` are properly styled. Tables should follow the

	header1 header2 format.\n\n**Important:** All links must be tested before publication.\n\nSee email from Sarah for additional context: <email:thread-37292@example.com>\n\n# Task history: \n**> Tech writer**\nI've started a draft at docs.google.com/document/d/1a2b3c4d5e. Please review the structure before I continue.\n\n**> Developer**\nThere are 3 new endpoints: `GET /users`, `POST /users`, and `DELETE /users/{id}`. I've documented the parameters in Swagger.\n\n**> Project Manager**\nDon't forget to update the navigation menu at <nav:http://docs.example.com/navigation.xml>. This should be synchronized with the sitemap.\n\n**> QA**\nWill need to verify all examples work with our [test environment](https://test.api.example.com). I can help test the code samples.\n\n", "category": "formatting_challenge", "language_detected": "English"}\n {"text": "# Task subject: Schedule quarterly team meeting\n\n# Task details: * Start date: 2025-02-01\n* Due date: 2025-02-15\n* Priority: Low\n* Status: Realisation\n\n# Task description: \nNeed to organize **Q2 planning meeting** for the engineering team. Should include project updates, roadmap review, and team building activity.\n\n* Duration: 2 hours\n* Required attendees: All engineering staff + product managers\n* Prepare agenda with time allocations\n\n\n# Task history: \n**> Manager**\nPlease find a time when the remote team can join. Preferably between 10 AM - 2 PM CET to accommodate different time zones.\n\n**> HR**\nWe can provide budget for lunch if the meeting is held on-site. Let me know the final headcount.\n\n**> Admin**\nConference Room B is available most Thursdays. The room can accommodate up to 25 people and has video conferencing equipment.\n\n", "category": "short_context", "language_detected": "English"}\n {"text": "# Task subject: TBD - project alpha\n\n# Task details: * Start date: 2025-03-01\n* Due date: \n* Priority: Normal\n* Status: To check\n\n# Task description: \nDetails to be confirmed. Placeholder for upcoming work.\n\n\nPriority may change.\n\n\n# Task history: \n**> Manager**\nWaiting for client input. Should have more details next week.\n\n**> Sales**\nShould be finalized next week. Contract discussion ongoing.\n\n", "category": "edge_case", "language_detected": "English"}\n
English	
English	{"text": "# Task subject: Mise à jour du système de paiement\n\n# Task details: * Start date: 2025-01-10\n* Due date: 2025-03-31\n* Priority: High\n* Status: Passive\n\n# Task description: \nNous devons mettre à jour notre **système de paiement** pour inclure de nouvelles méthodes et améliorer la sécurité. \n\n**Points importants:**\n1. Ajouter le support pour Apple Pay et Google Pay\n2. Mettre à jour le protocole de sécurité vers PCI-DSS v4.0\n3. Améliorer l'interface utilisateur du processus de paiement\n\n\nDate limite: fin du trimestre.\n\nDocumentation technique disponible sur docs.payment.example/fr\n\n\n# Task history: \n**> Équipe technique**\nLa migration nécessitera une interruption de service de 2 heures. Nous suggérons de la planifier pendant le weekend.\n\n**> Marketing**\nNous devons communiquer ces changements à nos clients. Je vais préparer une campagne d'email.\n\n**> Sécurité**\nUne analyse de vulnérabilité sera nécessaire avant le déploiement. J'ai planifié cela avec notre équipe pour la semaine prochaine.\n\n", "category": "edge_case", "language_detected": "French"}\n
French	

English

```
{
  "text": "# Task subject: Update login page UI\n\n# Task details: *\nStart date: 2024-12-15\n* Due date: 2025-01-10\n* Priority: High\n* Status: Realisation\n\n# Task description: \nNeed to refresh the login\npage with **new company branding**. Use the new color scheme\n(#3A7BF2, #F5F9FF) and updated logo.\n\n* Implement responsive\ndesign for mobile devices\n* Update form validation messages\n* Add\n\"Remember me\" checkbox\n\nThe changes should be deployed by\nFriday.\n\n# Task history: \n**> Designer**\nI've uploaded\nnew assets to the shared folder. Please use the SVG version of the logo\nfor better scaling.\n\n**> Project Manager**\nThis is part of the Q1\nrebranding effort. Make sure it matches the new style guide at [brand\nguidelines](https://company.example.com/brand).\n\n**>\nDeveloper**\nWill need to update CSS variables. I've started working\non this in branch `feature/login-ui-update`.\n\n", "category":\n  "short_context", "language_detected": "English"
}
```

English

```
{
  "text": "# Task subject: Prepare quarterly newsletter content\n\n# Task details: *\nStart date: 2025-03-01\n* Due date: 2025-04-10\n* Priority: Normal\n* Status: New\n\n# Task description: \nIt's time to\nprepare our **Q2 2025 customer newsletter**. The newsletter should\ninclude:\n\n1. Product updates from Q1\n2. Upcoming features (see\nroadmap:\n[roadmap.example.com/public](https://roadmap.example.com/public\n))\n3. Customer spotlight: Acme Corp implementation (reference:\n<case-study:CS-2025-03>)\n4. Industry news and trends\n5. Tips &\ntricks section\n\nPlease follow our <brand:guidelines:v3.2> for all\ncontent. Images should be optimized for email (max 600px width,\n<100KB).\n\nHeader template: <html><body><div\nclass=\"header\">...</div></body></html>\n\nFooter must include\nunsubscribe link: [Unsubscribe]({UNSUBSCRIBE_URL})\n\nDistribution list: [Q2-Customers-2025]\n\nTarget delivery\ndate: April 15, 2025\n\n# Task history: \n**>\nMarketing**\nWe need to highlight the new mobile features. They\nwere our biggest release this quarter.\n\n**> Design**\nI've prepared\nimages at\n[design.example.com/newsletter/q2](https://design.example.com/ne\nwsletter/q2). Please use these for consistency.\n\n**> Legal**\nMake\nsure we include the required disclaimers:\n<text:legal/disclaimer/2025/q2.txt>. This is especially important for\nthe forecasting section.\n\n**> CEO**\nI'd like to include a personal\nnote this time - see my draft at <draft://messages/ceo/q2-2025>.\nPlease review and suggest edits if needed.\n\n", "category":\n  "formatting_challenge", "language_detected": "English"
}
```

Příloha B: Implementace Score Evaluation: score_evaluation.py

```
# Schéma pro evaluaci s hodnocením 0-1
from pydantic import BaseModel, Field

class ScoreEvaluation(BaseModel):
    score: float = Field(
        description="Score from 0 to 1 where 0 is the worst and 1 is the best. A score of 1 means "
        "that the response perfectly fits the criteria, whereas a score of 0 means that "
        "it did not fit the criteria at all."
    )
    reasoning: str = Field(
        description="Reasoning behind the score"
    )
```

Příloha C: Implementace evaluátora pro posouzení srozumitelnosti: summarize_clarity_evaluator.py

```
from langsmith import wrappers
from openai import OpenAI

from evals.read_assistant.evaluators.score_evaluation import ScoreEvaluation

openai_client = wrappers.wrap_openai(OpenAI())

def clarity_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí srozumitelnost a jasnost shrnutí."""
    print(f"clarity_evaluator OUTPUTS: {outputs.keys()}")
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing the clarity of summaries."},
            {"role": "user", "content": f"""
            Evaluate the CLARITY of the following summary.

            A score of 1 is best and means that the response fits the criteria, whereas a score of 0 is worst and means that it did not fit the criteria.

            Consider the following when evaluating clarity:
            - Is the summary well-structured and easy to understand? (higher score)
            - Is the language clear and precise? (higher score)
            - Are there any confusing elements or ambiguous statements? (lower score)
            - Does the summary follow a logical structure? (higher score)

            Original text:
            """
            }
        ]
    )
```

```

        {inputs["text"]}

        Summary:
        {outputs["output"]}
        """
    ],
    response_format=ScoreEvaluation,
)
return {"score": response.choices[0].message.parsed.score,
        "reasoning": response.choices[0].message.parsed.reasoning}

```

Příloha D: Implementace evaluátora pro posouzení věcné správnosti: summarize_correctness_evaluator.py

```

from langsmith import wrappers
from openai import OpenAI

from evals.read_assistant.evaluators.score_evaluation import ScoreEvaluation

openai_client = wrappers.wrap_openai(OpenAI())

# Evaluátor pro Correctness
def correctness_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí správnost shrnutí vzhledem k původnímu textu."""
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing the correctness of summaries."},
            {"role": "user", "content": f"""
                Evaluate the CORRECTNESS of the following summary relative to the original text.

                A score of 1 is best and means that the response fits the criteria, whereas a score of 0 is worst and means that it did not fit the criteria.

                Consider the following when evaluating correctness:
                - Does the summary contain factual errors or misrepresentations? (lower score)
                - Does the summary accurately represent the key information from the original text? (higher score)
                - Are there any inaccuracies or omissions that change the meaning? (lower score)

                Original text:
                {inputs["text"]}

                Summary:
                {outputs["output"]}
                """
            ]},
        response_format=ScoreEvaluation,
    )

```

```
)
return {"score": response.choices[0].message.parsed.score,
        "reasoning": response.choices[0].message.parsed.reasoning}
```

Příloha E: Implementace evaluátora pro posouzení stručnosti odpovědi: summarize_conciseness_evaluator.py

```
from langsmith import wrappers
from openai import OpenAI

from evals.read_assistant.evaluators.score_evaluation import ScoreEvaluation

openai_client = wrappers.wrap_openai(OpenAI())

# Evaluátor pro Conciseness
def conciseness_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí stručnost a výstižnost shrnutí."""
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing the
conciseness of summaries."},
            {"role": "user", "content": f"""
Evaluate the CONCISENESS of the following summary.

        A score of 1 is best and means that the response fits the
criteria, whereas a score of 0 is worst and means that it did not fit the
criteria.

        Check specifically:
        - Summary is limited to 200 characters as required
        - Has exactly THREE bullet points under Key points
        - No excessive information
        - Includes only essential information without unnecessary
details

        Original text:
        {inputs["text"]}

        Summary:
        {outputs["output"]}
        """}
        ],
        response_format=ScoreEvaluation,
    )
    return {"score": response.choices[0].message.parsed.score,
            "reasoning": response.choices[0].message.parsed.reasoning}
```

Příloha F: Implementace evaluátora pro posouzení informační retence odpovědi:

summarize_info_retention_evaluator.py

```
from langsmith import wrappers
from openai import OpenAI

from evals.read_assistant.evaluators.score_evaluation import ScoreEvaluation

openai_client = wrappers.wrap_openai(OpenAI())

# Evaluátor pro Information Retention
def info_retention_evaluator(outputs: dict, inputs: dict) -> dict:
    """Hodnotí, jak dobře jsou zachovány faktické informace v shrnutí."""
    response = openai_client.beta.chat.completions.parse(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "You are an objective evaluator assessing information retention in summaries."},
            {"role": "user", "content": f"""
            Evaluate the INFORMATION RETENTION of the following summary.

            A score of 1 is best and means that the response fits the criteria, whereas a score of 0 is worst and means that it did not fit the criteria.

            Consider the following when evaluating information retention:
            - Does the summary retain all critical information from the original text? (higher score)
            - Are important details or key points missing? (lower score)
            - Does the summary maintain the core message and essential context? (higher score)

            Original text:
            {inputs["text"]}

            Summary:
            {outputs["output"]}
            """}
        ],
        response_format=ScoreEvaluation,
    )
    return {"score": response.choices[0].message.parsed.score,
            "reasoning": response.choices[0].message.parsed.reasoning}
```

Příloha G: Hlavní skript pro spuštění evaluace: summarization_evaluator.py

```
import argparse
import os
from pathlib import Path

from dotenv import load_dotenv
from fastapi import FastAPI
from langsmith import Client

import settings as server_settings
from app_state import app_state
from chain_api_client import ChainApiClient
from create_app import create_app
from evals.read_assistant.evaluators.summarize_clarity_evaluator import clarity_evaluator
from evals.read_assistant.evaluators.summarize_conciseness_evaluator import conciseness_evaluator
from evals.read_assistant.evaluators.summarize_correctness_evaluator import correctness_evaluator
from evals.read_assistant.evaluators.summarize_info_retention_evaluator import \
    info_retention_evaluator
from routers.api import ApiGenerateRequest
from utils import sign_jwt

load_dotenv()

# Přístup k API klíčům
langsmith_api_key = os.getenv("LANGSMITH_API_KEY")
openai_api_key = os.getenv("OPENAI_API_KEY")
langsmith_tracing = os.getenv("LANGSMITH_TRACING")

def _read_prompt() -> str:
    """
    Vytvoří prompt pro generování shrnutí podle zadané struktury.
    """
    current_folder = Path(__file__).parent
    prompts_folder = current_folder / "prompts"
    with open(prompts_folder / "summarize_eval_prompt.md", "r",
encoding="utf-8") as f:
        content = f.read()
    return content

class SummarizationEvaluator:
    llm_id: str
    prompt: str
    app: FastAPI | None = None
    chain_api: ChainApiClient | None = None

    def __init__(self, llm_id: str):
        if not llm_id:
            raise ValueError("llm_id is required")

        self.llm_id = llm_id
        self.prompt = _read_prompt()
```

```

self.ls_client = Client()

def process_example(self, inputs: dict) -> dict:
    """
    Funkce, kterou budeme evaluovat. Přijímá vstupy z datasetu
    a vrací odpověď modelu.
    """

    request = ApiGenerateRequest(
        prompt_template=self.prompt,
        input_variables={
            "text": inputs["text"],
            "language": inputs["language_detected"],
        }
    )

    response = self.chain_api.generate(request=request)
    return {"output": response.output}

def run(self):
    server_settings.JWT_SECRET_KEY = "JWT_SECRET_KEY"
    server_settings.JWT_ALGORITHM = "HS256"
    token = sign_jwt("test_user")

    self.app = create_app(app_state=app_state, unit_test=True)

    self.chain_api = ChainApiClient(app=self.app, jwt_token=token)

    llm_registry = app_state.llm_registry
    provider = llm_registry.get_provider(llm_id=self.llm_id)
    model_name = provider.model_name

    metadata = {
        "llm_id": self.llm_id,
        "model_name": model_name,
        "evaluator_llm_id": "gpt-4o",
        "evaluator_model_name": "gpt-4o",
    }

    # Fetch the existing dataset
    all_datasets = self.ls_client.list_datasets()
    all_datasets = list(all_datasets)
    dataset_name = "Easy AI Read Assistant - Summarize 2.0"
    my_datasets = list(filter(lambda ds: ds.name == dataset_name,
all_datasets))
    if len(my_datasets) == 0:
        raise ValueError(f"No datasets {dataset_name} found")
    if len(my_datasets) != 1:
        raise ValueError(f"Multiple datasets with name {dataset_name}")
    dataset = my_datasets[0]

self.ls_client.read_dataset(dataset_id=dataset.id)
    print(f"Using dataset '{dataset.name}' with ID: {dataset.id}")

    print(f"Starting evaluation for dataset {dataset.name}...")

    experiment_results = self.ls_client.evaluate(
        self.process_example,
        data=dataset.id,
        evaluators=[
            correctness_evaluator,
            conciseness_evaluator,

```

```

        info_retention_evaluator,
        clarity_evaluator
    ],
    experiment_prefix="summary_prompt_evaluation",
    max_concurrency=2,
    metadata=metadata
)

print(f"Results: {experiment_results}")

if __name__ == "__main__":
    arg_parser = argparse.ArgumentParser()
    arg_parser.add_argument("--llm-id", type=str, required=False,
default="gpt-4o")
    args = arg_parser.parse_args()

    evaluator = SummarizationEvaluator(llm_id=args.llm_id)
    evaluator.run()

```